Final Year Project

MediSim



Bachelor's Studies Information Technology (2022-2026)

Submitted To

Faculty of Computing &
Information Technology (FCIT),
University of the Punjab, Lahore, Pakistan.

Project ID: GGCC-GRW-FYP22-06

Under the Supervision of
Prof. Waqas Jamil Mughal

By

Umais Ur Rehman	**075943**
Muhammad Zubair	**075935**
Moeed Ali	**075944**

GOVT. GRADUATE COLLEGE OF COMMERCE
PEOPLE COLONY, GUJRANWALA.

Final Year Project

MediSim



Project Supervisor:

Prof. Waqas Jamil Mughal

SUMBITTED BY:

Umais Ur Rehman	**075943**
Muhammad Zubair	**075935**
Moeed Ali	**075944**

Bachelor's Studies Information Technology (2022-2026)

PRESENTED TO:

Faculty of Computing &
Information Technology (FCIT),
University of the Punjab, Lahore, Pakistan.

SESSION: 2022-2026

GOVT. GRADUATE COLLEGE OF COMMERCE
PEOPLE COLONY, GUJRANWALA.

FYDP Overview

FACULTY OF COMPUTING & INFORMATION TECHNOLOGY
(FCIT) UNIVERSITY OF THE PUNJAB, LAHORE
FINAL YEAR DESIGN PROJECT (FYDP) REGISTRATION FORM

SEMESTER FALL 2025

FYDP Title: MediSim

FYDP Advisor

Name: Prof. Waqas Jamil Mughal **Signature**_____
Address: Govt. Graduate College of Commerce, Gujranwala
Email: _____ **Mobile:**_____

Sr. #	Roll Number	Team Member Name	Signature
1	075943	Umais Ur Rehman	
2	075935	Muhammad Zubair	
3	075944	Moeed Ali	

FYDP Goals	Deliver a fully operational MediSim system prototype.
FYDP Objective	• Develop a scalable Mobile Application using python framework **Fast API (Back-End)** and **Flutter (Front-End).** • Implement **secure authentication** using Email-based OTP verification. • Provide a **simulation graph** for medical training students including a graph simulation, vital values, quiz system and notification. • Enable **advance platform features** including real time simulation, ai based notification and ai chat bot.
FYDP Scope	The scope of the MediSim project is designed to medical training environments by providing a smart and user-friendly medical simulation system. The system will handle essential functions such as Simulation Graph, Quiz System, AI Chat Bot, Generate Performance Report. It will help reduce the reading books and physical medical situation for medical student during training sessions. The system also provides a simple and interactive interface that manage medical training.

(Ch. M. Afzal) Principal

Statement of Submission

This is certified that **Umais Ur Rehman**, **Muhammad Zubair** and **Moeed Ali** has successfully completed the documentation of their final year project at **Govt. Graduate College of Commerce, Gujranwala** as partial fulfillment for the degree of **Bachelor's Study in Information Technology.**

Signature……...

Supervisor

Prof. Waqas Jamil Mughal.

Lecturer Govt, Graduate College of Commerce.

Signature…….

HOD of IT Department

Prof. Nadeem Ahmad Naz,

Govt Graduate College of Commerce.

Signature……...

External Examiner

Signature…….

Principal

Ch. Muhammad Afzal

Govt Graduate College of Commerce.

Certificate of Approval

It is to certify that the final year design project (FYDP) of **BSIT "MediSim**" was developed by **Umais Ur Rehman (075943), Muhammad Zubair (075934)** and **Moeed Ali (075944)** under the supervision of "**Prof. Waqas Jamil Mughal**" in my opinion. it is fully adequate, in scope and quality for the degree of Bachelors of Science in **Information Technology**.

Signature----------

FYDP Supervisor:

Signatures (Faculty Advisory Committee (FAC):

Signature			
Name			
	FAC1	FAC2	FAC3

Signature:
Head of FYDP Coordination Office:

Signature---------- Dated----------

Chairperson, Department of Information Technology

Executive Summary

MediSim is a digital learning tool designed for medical students and healthcare trainees. They can practice clinical skills in a safe environment without any risk to a real patient. The idea is straightforward: the student, or trainee, can use the mobile device to put themselves in realistic clinical scenarios, to enhance their learning, make their mistakes, and develop their skills before working with real patients. But medical simulations are challenging. MediSim solves this problem by giving students a place to practice as many times as they want, with no real-world consequences.

The system provides different user roles like instructor and learner. The instructor builds and maintains the medical scenarios and the learner participates in the simulation to improve their skills. MediSim uses a use-case based approach to clearly define how users interact with the system to be easily understood and used. The functional requirements include user registration and login, scenario selection, real time patient monitoring, vital signs and performance feedback, quiz system, ai chat bot. The non-functional requirements are related to the system performance, security, reliability and usability to ensure smooth operation and user-friendly experience.

Acknowledgement

We are grateful to the **Almighty Allah**, who blessed me with health, wisdom, knowledge, thoughts and opportunity to make some contribution in the form of present effort. We offer my humblest thanks from the deepest core of my heart to the Holy Prophet Muhammad (Peace be upon him), the most perfect and excelled among and ever born on the surface of earth. The research work embodied in this dissertation was accomplished under the able guidance and affectionate supervision of **Prof. Habib-Ullah, Prof. Nadeem Ahmad Naz.** and **Prof, Waqas Jamil Mughal**. We will always remember his moral encouragement, skillful guidance, positive criticism and valuable advice throughout the course of my study. We also express my feelings of love and respect for my beloved parents that are the real asset of my life and gave me confidence and my friends also deserve my thanks for their loving encouragement and prayers for my success.

Signature: ----------

Umais Ur Rehman (075943)

Signature: ----------

Muhammad Zubair (075934)

Signature----------

Moeed Ali (075944)

Abbreviation

Abbreviation	Description
EMS	Emergency Medical Services
ECG	Electrocardiogram
SPO2	Peripheral Oxygen Saturation
BP	Blood Pressure
ETCO2	End-Tidal Carbon Dioxide
HR	Heart Rate
RR	Respiratory Rate
AI	Artificial Intelligence
UML	Unified Modeling Language
ERD	Entity Relationship Diagram
MVC	Model View Controller
API	Application Programming Interface
UI	User Interface
UX	User Experience
FYP	Final Year Project
FCIT	Faculty of Computing and Information Technology
SRS	Software Requirement Specification
SDS	Software Design Specification
OOP	Object Oriented Programming

SQL	Structured Query Language
MySQL	My Structured Query Language
DBMS	Database Management System
QR	Quick Response
OTP	One-Time Password
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
REST	Representational State Transfer
SDK	Software Development Kit
IDE	Integrated Development Environment
CRUD	Create Read Update Delete
RBAC	Role-Based Access Control
GUI	Graphical User Interface
API Endpoint	Backend URL used for communication
Session ID	Unique identifier for simulation session
Authentication	Process of verifying user identity
Authorization	Process of granting access permissions
Real-Time Sync	Instant synchronization of data
Adaptive Quiz	Quiz that changes difficulty dynamically

Table of Content

Chapter 1: Introduction **1**

1.1 Introduction 2

1.2 Background 2

1.3 Problem Statement 3

1.4 Problem Solution 3

1.5 Scope 3

1.6 Vision Statement 3

1.7 Related System Analysis and Literature Review 4

1.7.1 Resus Monitor 4

1.7.2 Simpl-Simulated Patient Monitor 4

1.8 Tools and Technologies 4

1.9 Project Deliverables 5

1.10 Project Planning 5

1.11 Assumptions 5

1.12 Work Division 6

1.13 Summary 6

Chapter 2: Requirement's Analysis **7**

2.1 Analysis 8

2.2 User Classes and Characteristics 8

2.3 Requirement Identifying Technique 8

2.4 Functional Requirements 11

2.4.1 Login/Registration 12

2.4.2 Emergency Scenarios 13

2.4.3 Vital Graph Simulations 14

2.4.4 Adaptive Quiz System 15

2.4.5 AI Chat Bot 16

2.4.6 Share Vital values using ID and QR Code 17

2.4.7 Skill Scoring and AI Generated Reports 18

2.4.8 Interesting Notifications 19

2.4.9 Change Vital Values 20

2.5 Non-Functional Requirements 21

2.5.1 Reliability 21

2.5.2 Performance 21
2.5.3 Visual Quality 21
2.5.4 Accuracy 21
2.5.5 User Experience 21
2.6 Use Case Analysis 22
2.6.1 Register User 22
2.6.2 Select Simulation Scenario 23
2.6.3 AI Chat Bot 24
2.6.4 Generate Performance Report 25
2.6.5 Adaptive Quiz System 26
2.6.6 Notification System 27
2.6.7 Skill Scoring and Performance Monitoring 28
2.6.8 Session Creating and QR Code and ID 29
2.6.9 Change Vital Values 30
2.7 Summary 31
Chapter 3: Design and Architecture **32**
3.1 System Design 33
3.2 Design Consideration 33
3.3 Design Model 33
3.3.1 Context Diagram 34
3.3.2 Design Class Diagram 35
3.3.3 Sequence Diagram 37
3.3.4 State Transition Diagram 44
3.4 Architectural Design 46
3.4.1 UML Class Relationship Diagram 46
3.4.2 UML Component Diagram 48
3.4.3 Layer Architecture 49
3.5 MVC 49
3.6 Data Design 49
3.6.1 ER Diagram 50
3.6.2 MYSQL Data Structure 52
3.6.3 Data Processing and Organization 53
3.6.4 Data Dictionary 53
3.6.5 Functions and Parameter 57

3.6.6 Object Oriented Approach 58
3.7 User Interface Design 62
3.7.1 Screen Images 63
3.8 Behavioral Model 66
3.9 Design Decision 67
3.10 Summary 68
Chapter 4: Implementation **69**
4.1 Implementation 70
4.1.1 Algorithm 70
4.2 External APIs/SDKs 79
4.3 Code Repository 79
4.3.1 Git Repository Link 80
4.4 Summary 80
Chapter 5: Testing and Evaluation **81**
5.1 Testing Evaluation 82
5.2 Unit Testing 82
5.3 Integration Testing 87
5.4 System Testing 90
5.5 Performance Testing 93
5.6 Summary 96
Chapter 6: System Conversion **97**
6.1 System Conversion 98
6.2 Conversion Method 98
6.2.1 Direct Conversion 98
6.2.2 Parallel Conversion 98
6.2.3 Phased Conversion 98
6.2.4 Pilot Conversion 98
6.3 Deployment 99
6.4 Data Conversion 100
6.5 Training 101
6.6 Post Deployment Test 102
6.7 Challenges 102
6.8 Summary 102
Chapter 7: Conclusion **103**

7.1 Conclusion 104
7.2 Evaluation 104
7.3 Traceability Matrix 105
7.4 Future Work 105
Appendix A **106**
Use Case Analysis 107
Appendix B **112**
Coding Standards **112**
1. Variable Naming Conventions 113
2. Function Naming 113
3. Code Comments 113
4. Indentation and Formatting 113
5. File and Foder Naming 113
6. Code Structure and Modularity 114
7. Access Modifier 114
9. Exception Handling 114
Appendix C **115**
Application Prototype **115**
1. Login Screen 117
2. Simulation Graphs 118
3. AI Chat Bot 119
4. Performance Report 120
5. QR Code and ID 121

List of Figure

Figure 1: Gantt Chart 5
Figure 2: Use case Diagram 10
Figure 3: Context Diagram 34
Figure 4: Design Class Diagram 36
Figure 5: Sequence Diagram for Login and Registration 38
Figure 6: Sequence Diagram for View Graphs 39
Figure 7: Sequence Diagram for Quiz and Performance 40
Figure 8: Sequence Diagram for AI Chat Bot 41
Figure 9: Sequence Diagram for Session ID and QR Code 42
Figure 10: Sequence Diagram for Notification System 43
Figure 11: State Transition Diagram 45
Figure 12: UML Class Relationship Diagram 47
Figure 13: UML Component Diagram 48
Figure 14: ER Diagram 51
Figure 15: Login Screen and Share QR Code 63
Figure 16: View Graphs and Scenarios 64
Figure 17: AI Assistant and Progress Screen 65
Figure 18: Behavioral Model 66

List of Tables

Table 1 Tools and Technologies 4
Table 2 Work Division 6
Table 3 User Classes and Characteristics 8
Table 4 Login/Registration 12
Table 5 Emergency Scenario 13
Table 6 Vital Graph Simulations 14
Table 7 Adaptive Quiz System 15
Table 8 AI Chat Bot 16
Table 9: Share Vital values using ID and QR Code 17
Table 10 Skill Scoring and AI Generated Reports 18
Table 11 Interesting Notification 19
Table 12 Change Vital Values 20
Table 13 Use Case # 1: Register User 22
Table 14 Use Case # 2 Select Simulation Scenario 23
Table 15 Use Case # 3 AI Chat Bot 24
Table 16 Use Case # 4 Generate Performance Report 25
Table 17 Use Case # 5: Adaptive Quiz System 26
Table 18 Use Case # 6: Notification System 27
Table 19 Use Case # 7: Skill Scoring and Performance Monitoring 28
Table 20 Use Case # 8: Session Creating and QR Code 29
Table 21 Use Case # 9: Change Vital Values 30
Table 22 Layer Architecture 49
Table 23 MVC Component 49
Table 24 Data Dictionary 53
Table 25 Functions and Parameter 57
Table 26 Login/Registration 71
Table 27 Vital Graph Simulation 72
Table 28 Adaptive Quiz System 73
Table 29 AI Chat Bot 74

Table 30 Share Vital via QR Code and ID 75
Table 31 Skill Scoring and AI Generated Reports 76
Table 32 Interesting Notification 77
Table 33 Change Vital Values 78
Table 34 External APIs/SDKs 79
Table 35: Repository Detail 80
Table 36 UT1: Login / Registration 82
Table 37 UT2: Emergency Scenario 83
Table 38 UT3: Vital Graph Simulation 83
Table 39 UT4: Change Vital Values 84
Table 40 UT5: Adaptive Quiz System 84
Table 41 UT6: Skill Scoring and AI Report 85
Table 42 UT7: AI Chat Bot 85
Table 43 UT8: Share Vital via QR Code and ID 86
Table 44 UT9: Interesting Notification 86
Table 45 ITC1: Login / Registration 87
Table 46 ITC2: Emergency Scenario 87
Table 47 ITC3: Vital Graph Simulation 87
Table 48 ITC4: Change Vital Values 88
Table 49 ITC5: Adaptive Quiz System 88
Table 50 ITC6: Skill Scoring and AI Report 88
Table 51 ITC7: AI Chat Bot 89
Table 52 ITC8: Share Vital via QR Code and ID 89
Table 53 ITC9: Interesting Notification 89
Table 54 ST1: User Registration 90
Table 55 ST2: Simulation Scenario 90
Table 56 ST3: Vital Graph 90
Table 57 ST4: Change Vital Values 91
Table 58 ST5: AI Chat Bot 91
Table 59 ST6: Adaptive Quiz System 91
Table 60 ST7: Skill Scoring and AI Report 92

Table 61 ST8: Session Creation 92
Table 62 ST9: Share Vital via QR Code and ID 92
Table 63 ST10: Notification System 93
Table 64 PT1: User Registration 93
Table 65 PT2: User Activation 93
Table 66 PT4: Vital Graph 94
Table 67 PT4: Chat Bot Performance 94
Table 68 PT5: Quiz Performance 94
Table 69 PT6: QR Code and ID Sharing 95
Table 70 PT7: Report Creation 95
Table 71 PT8: Joining Sessions 95
Table 72 PT9: Change Vital Values 96
Table 73 Android Deployment Steps 99
Table 74 Docker Deployment 100
Table 75 Data Conversion Plan 101
Table 76 Evaluation 104
Table 77 Traceability Matrix 105

Chapter 1: Introduction

1.1 Introduction

This app is made for EMS (Emergency Medical Services) students to help them learn in a practical way. It shows patient vital signs in real time with simulated waveforms. Through this app, students can practice reading vital signs and better understand how patients react in different situations. In emergency medical situation it is necessary to accurately interpret ECG, SPO2, BP (Blood pressure Blood pressure), and ETCO2 for making the timely and precise decisions.

The simulator depicts the real time dynamic nature of patient vitals like, which allows the students to observe continues changes in heart rate, oxygen saturation, blood pressure as they are designed to simulate emergency medical scenarios, Students can practice identifying abnormal curves in the graphs of the vitals and making critical predictions without the risk of real-world consequences.

Moreover, this app includes the lesson-based chapters for EMS students including basic vital signs, Electrocardiography, blood oxygen level, and BP. Each lesson is designed to build the students' knowledge. After every lesson Students take quizzes generated by AI focused on topics covered in the lesson. The quiz results contribute to skill scoring, which generates the student's mastery of the topic.

1.2 Background

Medical education requires strong practical training, especially for EMS students, but real-life clinical exposure is often limited due to patient safety concerns and lack of resources. To solve this problem, MediSim is a digital learning tool designed for medical students and healthcare trainees. They can practice clinical skills in a safe environment without any risk to a real patient. The premise is simple — let learners experience real-life medical situations on a mobile device so they can learn, make mistakes and improve before they work with real patients. Medical simulations are not easy to learn. Students need a lot of practice before they feel ready. But practicing on actual patients too soon can be hazardous. MediSim solves this problem by giving students a place to practice as many times as they want, with no real-world consequences.

1.3 Problem Statement

The biggest challenge for EMS students is that just reading books will not teach them how to save lives. Students today see static pictures of blood pressure or heart rates in their textbooks. But in a real medical emergency, those signs are always changing. It is very difficult for a student to make a fast decision when they see a moving line on an ECG monitor for the first time. And we can't practice on real patients because one mistake could kill someone. Students get scared and lose confidence when they see a real patient as they don't have a safe method to practice.

1.4 Problem Solution

Our We are developing an app called MediSim that functions on a phone like a genuine patient to address these problems. Students can now view live, moving lines on their displays rather than merely reading about blood pressure or heart rates in outdated literature. This gives the impression that they are in a genuine ambulance throughout the entire encounter. The fact that it's a secure learning environment is crucial. No one is harmed if a learner makes a mistake or hits the incorrect button. Until students are no longer afraid and feel prepared for the actual world, they can repeatedly attempt the same emergency scene. Additionally, we included an intelligent AI into the app to grade students' work and provide quizzes

1.5 Scope

This project is about an AI based EMS (Emergency Medical Services) training app which helps the medical students in analyzing the vital graphs like ECG, SPO2, BP, and ETCO2 graphs and learning how to make decisions in an emergency situation. The app combines different simulation scenarios of graph, AI assisted guidance, and interactive learning for the students.

1.6 Vision Statement

Our primary objective is to give EMS students an app that mimics real life to transform the way they learn. In the future, we hope that no medical student will be afraid or perplexed when they first see an oxygen monitor or a moving heartbeat line. Our goal with MediSim is to ensure that every student can safely practice saving lives without worrying about causing harm to others. Our goal is to bridge the gap between reading large books and working in an actual emergency. We think that by utilizing AI and moving graphs, we can help students become more self-assured and learn how to make quick, wise decisions.

1.7 Related System Analysis and Literature Review

1.7.1 Resus Monitor

In the field of simulation and training systems many solutions have been created to help students and professionals understand patient monitoring systems. Two known systems are Resus Monitor and Simpl Patient Monitor App, which offer simulation-based learning environments.

Resus Monitor is a web application that provide a different simulation graph of scenario. They have a limited simulation graphs and not support the guidance. But it lacks structured learning support such as a, quizzes, AI guidance or performance and slow down system. This limits its effectiveness as an educational tool. It costs nothing.

1.7.2 Simpl-Simulated Patient Monitor

The Simpl (Simulated Patient Monitor) App is a mobile application that simulates basic patient monitoring scenarios. However, the app only offers training features and doesn't include. structured learning support such as a, quizzes, AI guidance or performance and slow down system. It also is. Paid or has limited access, which may restrict its usefulness for a broader audience. Though, it's a great platform to learn but it doesn't have free usage for the students.

1.8 Tools and Technologies

Table 1 Tools and Technologies

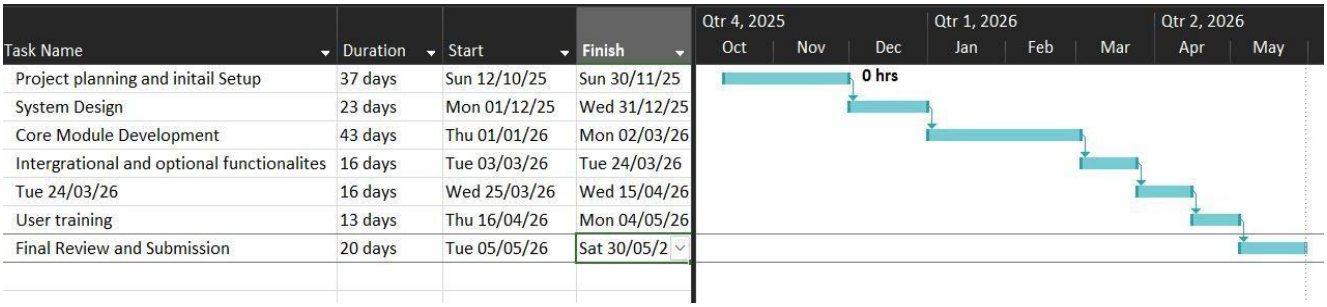
	Tools	**Version**	**Rationale**
Tools, Libraries, And Technologies	Android Studio	Latest	Offers fast rendering and excellent user experience.
	Dart Language	Latest	Provides scalability and real-time event handling.
	MY SQL	Latest	Real-Time Database Capabilities, Scalability, Integrated Authentication.
	Flutter	Latest	For Cross-Platform

1.9 Project Deliverables

- Project Proposal
- SRS
- SDS
- Final Year Project (D1) including FIGMA Design
- Final Year Project Report
- Application Deployment (GitHub)

1.10 Project Planning

Figure 1: Gantt Chart



1.11 Assumptions

- User must be own a smart mobile device.
- User must be able to read and understand English.
- User must have a valid email address for registration and login.
- User must have basic knowledge of the medical field or must have an instructor.
- User will use the system only for educational purpose only not on real patients.

1.12 Work Division

Table 2 Work Division

Sr. No	Roll Number	Name	Role Assignment & Work Division
1.	075943	Umais Ur Rehman	**Backend Development:** He builds main logic of the system so that can handle ai chat bot, and correctly moving in real-time simulation graphs, vital signs.
2.	075935	Muhammad Zubair	**Database Design:** He organizes how the app saves all the lessons, the AI-made test questions, and the students' final scores.
3.	075944	Moeed Ali	**Frontend and Documentation:** Manages the GUI design, user interface features and preparing technical documentation and reports.

1.13 Summary

MediSim transform traditional reading books and practice in critical situation and lost confident and manual effort. With a user-friendly interface, real time simulation, and Every function performs instantly without any delay. This system is designed for run every mobile device with minimal but powerful, versatile programming technologies. By bridging the gaps in existing system, MediSim offers a modern, reliable, and future ready solution for better Simulation System.

Chapter 2: Requirement's Analysis

2.1 Analysis

EMS students have a training app called MediSim that will enable them to practice medical skills on their mobile phone. The app recreates a life like ambulance and presents them with live vitals data such as, heart rhythm. Students will be able to observe how vital signs change during a clinical event without placing a real life human at risk. This ability to make and learn from mistakes is what will give EMS students confidence when placed in a real clinical situation.

The app does not run simulations but leads the student through structured learning units and practice units. They learn these units at their own speed and each unit builds on the knowledge that was attained from the one previous. It includes a knowledgeable test that builds off of each EMS course unit that will enable the learner to determine where they are excelling and where they need to study more.focus.

2.2 User Classes and Characteristics

Table 3 User Classes and Characteristics

User Classes	**User Characteristics**
Students	The students are the trainees that the app will be used for practicing medical scenarios and seeing health graphs live. The idea is to help students to learn through the simulated graph, clear doubts with the help of AI chat, learn from their mistakes without taking any risk, prepare tests, and build confidence for work in real emergency situations.
Instructor	The instructors are teachers who use the app in order to Create and manage the sessions of learning the graphs. Instructors can view the students' performance and their grades through a progress report.

2.3 Requirement Identifying Technique

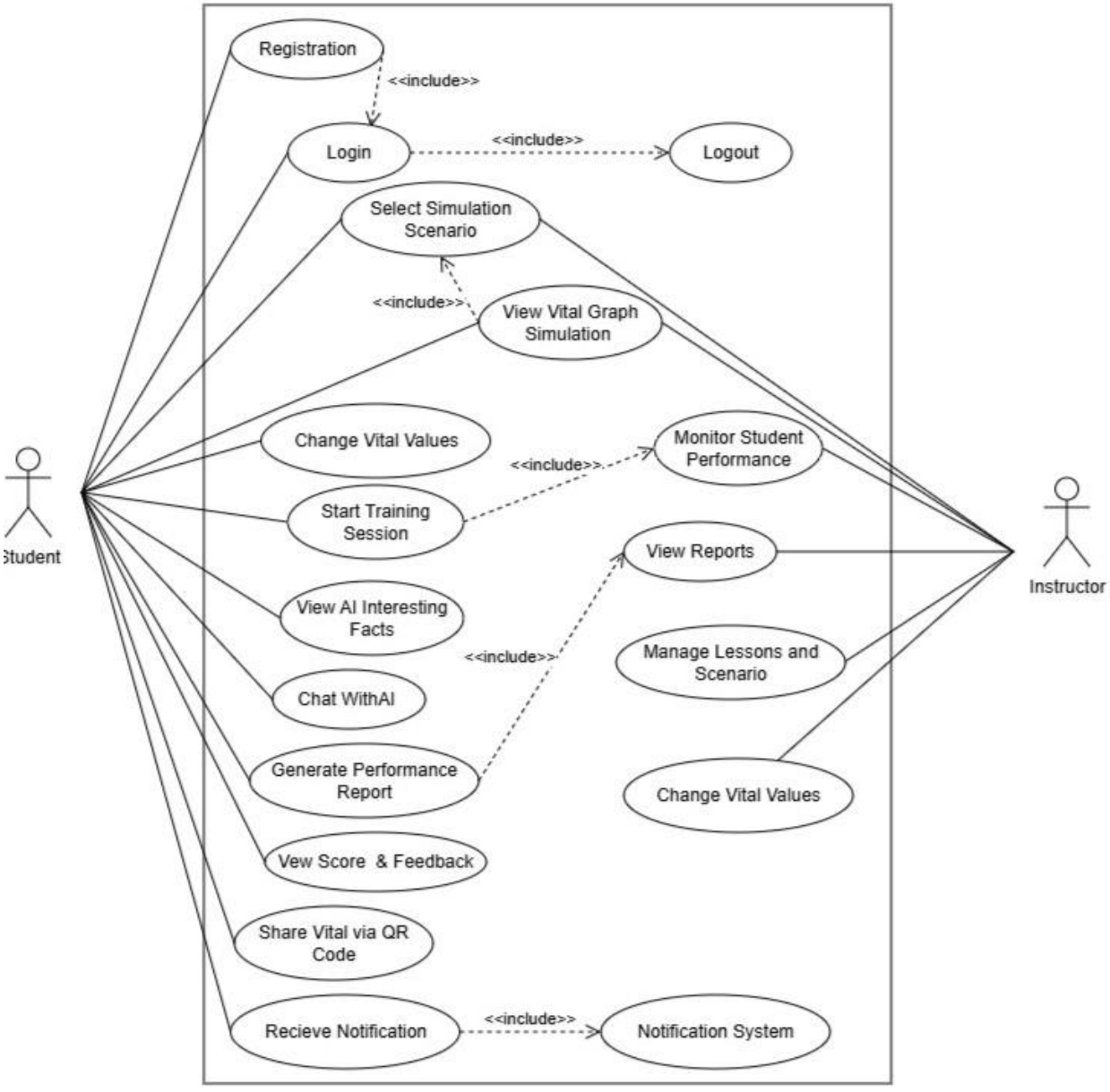
We are going to use a **Use Case Model** to define precisely how the MediSim application will function. A Use Case Model will allow us to view the overall concept, of how people will use the MediSim application to teach and learn from their trainees. It can also be easily customized for how each feature will relate to the user, without being overloaded with complicated details.

- There are two parts to the model:

  The Use Case Diagram: The shape represents the function in the application and the people who interact (students, teacher) with them.
- **The Use Case Analysis:** Shows what the user needs to do, what will occur on the screen and how the task is successfully completed.

Use Case Diagram

Figure 2: Use case Diagram



2.4 Functional Requirements

Functional Requirements are just what the MediSim app must do. Every aspect of the system was designed with teachers and students in mind and therefore the student and teacher will both find the system easy to use. The app provides real-time medical data such as heart rhythms like you will find in a hospital environment so the students have something that is truly reflective of real patient encounters. This feature is supplemented by structured learning and tests that assess the learning of each student over time so teachers can easily identify individual progression and weak points for individual students. This way, when the students enter their first real patient situation, they are much more comfortable. Due to the fact that every aspect of the system was built around the application, each feature of the app has a specific purpose and the user interface is easy to use without feeling tacked on or unnatural.

2.4.1 Login/Registration

Table 4 Login/Registration

Identifier	MSFR-1
Title	Login/Registration
Requirement	System should allow users to create a new account by specifying an email and password. System should allow users to login. System should allow users to logout.
Source	Project Requirements, Students, and Teachers
Rationale	To let users get into the app securely so they can save their progress, track their learning, and see their own personal scores.
Business Rule	Your e mail must be different from the ones previously inserted. (one per account only). Your password has to be secure to keep the data.
Dependencies	None
Priority	High

2.4.2 Emergency Scenarios

Table 5 Emergency Scenario

Identifier	MSFR-2
Title	Emergency Scenarios
Requirement	Users should be able to choose one medical scenario from a list (based on vital values). After choosing live ECG rhythms should be displayed.
Source	Project Requirements, Medical Experts, and Instructors
Rationale	To provide a hands-on way for students to practice treating different illnesses and improve their decision-making skills in a safe environment.
Business Rule	A student can only start a new scenario after finishing the current one or hitting the reset button. The data shown must match real-world medical symptoms for that specific case.
Dependencies	MSFR-1 (User must be logged in to access practice cases
Priority	High

2.4.3 Vital Graph Simulations

Table 6 Vital Graph Simulations

Identifier	MSFR-3
Title	Vital Graph Simulation
Requirement	The application should have live graphs displaying. Heart Rate, Spo2, Blood Pressure and EtCO2. The graph should be responsive with vitals value.
Source	Project Requirements, User need and Training goal
Rationale	To let students practice watching patient signs in a safe way so they feel ready for real-life hospital work.
Business Rule	All graphs must look clear and move smoothly. The data should be accurate so that students learn the right medical signs.
Dependencies	Login/Registration (MSFR-1)
Priority	High

2.4.4 Adaptive Quiz System

Table 7 Adaptive Quiz System

Identifier	MSFR-4
Title	Adaptive Quiz System
Requirement	The app create adaptive tests for the students. The questions will not be fixed; they will be chosen randomly by the system.
Source	Project Requirements, Teachers and the Students.
Rationale	Teachers and the Project Team
Business Rule	The app needs a huge bank of different questions. It should never feel like a one size fits all tests.
Dependencies	Login/Registration (MSFR-1)
Priority	High

2.4.5 AI Chat Bot

Table 8 AI Chat Bot

Identifier	MSFR-5
Title	AI Chat Bot
Requirement	It will feature an AI chatbot where students can type their question during a practice session. It is crucial for the AI to provide fast response about medical related.
Source	Project Requirements, Students need and Project Team
Rationale	To give students a way to get help instantly without needing a teacher always standing over them.
Business Rule	I only want the AI to speak about medical issues relevant and providing clear, brief, actionable advice.
Dependencies	Login/Registration (MSFR-1)
Priority	Medium

2.4.6 Share Vital values using ID and QR Code

Table 9: Share Vital values using ID and QR Code

Identifier	MSFR-6
Title	Share Vital via QR Code and ID
Requirement	The app generates a scannable QR Code and ID that holds the vital signs and send to other students to scan the QR Code or ID to view data quickly.
Source	Project Requirements, The Project Team and Students
Rationale	To help students share data quickly during a busy practice session without having to type everything in by hand.
Business Rule	The QR code must be easy to read with a phone camera. It should only show the data from the current medical session.
Dependencies	Vital Graph Simulation (MSFR-2)
Priority	Medium

2.4.7 Skill Scoring and AI Generated Reports

Table 10 Skill Scoring and AI Generated Reports

Identifier	MSFR-7
Title	Skill Scoring and AI Generated Report
Requirement	The app has to calculate the score after every Quiz. The app present a detailed report with students' s performance and check the graph simulation presentation.
Source	Project Requirements, Training Goals and Project Needs
Rationale	To give students instant feedback so they can fix their mistakes and get better at medical tasks right away.
Business Rule	Grading must be fair and based on real medical rules. The report should be easy to read and available to download as a file.
Dependencies	Progress Tracking (MSFR-3)
Priority	High

2.4.8 Interesting Notifications

Table 11 Interesting Notification

Identifier	MSFR-8
Title	Interesting Notification
Requirement	The app sends a notification of related medical knowledge, different simulation scenarios and interesting facts.
Source	Project Requirements, Students and Teachers
Rationale	To keep the students interested in the app and help them learn small facts in their free time.
Business Rule	Notifications shouldn't pop up too many times or they will become annoying. They must be short and easy to read.
Dependencies	User Login (MSFR-1)
Priority	Low

2.4.9 Change Vital Values

Table 12 Change Vital Values

Identifier	MSFR-9
Title	Change Vital Values
Requirement	The instructor and user allow the change of any different graph simulation and vital values and can change.
Source	Project Requirements, Students and Instructor
Rationale	This feature is the enables simulation scenarios by allowing dynamic changes and helping students learn how to respond to different medical situations.
Business Rule	Only authorized users and Instructor can change vital value.
Dependencies	User Login (MSFR-1)
Priority	High

2.5 Non-Functional Requirements

2.5.1 Reliability

The system has to be reliable-the goal is 99.9% of the time is the system to be up and not producing false or errors. The system should run continually without freezing or crashing during those long training sessions we mentioned earlier on. Unit tests, integration tests, and performance tests should be continually conducted and logged to ensure we are catching bugs earlier rather than later.

2.5.2 Performance

It should show graphs of the heart rate and blood pressure in real-time with no delay. While the student is operating it, the line on the graph and the values should move just like real monitors without a jump or skip. Any slight freeze up or pause will defeat the purpose. The app has to be quick enough to keep up with all the inputs.

2.5.3 Visual Quality

It all needs to look sharp and clean. Heart graphs have tons of tiny details and the students should be able to see it clearly. There's no point in the screen being fuzzy or having too tiny of numbers; especially under a pressure situation. Therefore, we are keeping it simple but professional.

2.5.4 Accuracy

We value your accuracy. Any vital sign or graph within the app is actual medical data and not random. The app has been thoroughly tested to ensure the app remains stable even when used over a long duration. This process is to test for any errors that might have slipped in and fix them right away.

2.5.5 User Experience

MediSim is about an experience that is easy and no difficulty. When the student clicks on any button, the real-time vital signs/graphs pop up instantly and react to what the student clicks. There's no irritating delay. The focus is on allowing students to learn without any stress or extra work. In the end, it’s about helping students focus, practice, and learn in a comfortable environment.

2.6 Use Case Analysis

2.6.1 Register User

Table 13 Use Case # 1: Register User

UC Identifier	MS-UC1
UC Name	Register User
Requirements Traceability	MSFR-1
Purpose	To allow students to create a personal account to save their training progress.
Priority	High
Preconditions	The user must be on the sign-up screen.
Post Conditions	The user is registered and can now log in to the app.
Actors	Student / Instructor
Extends	None
Main Success Scenario	The user arrives at the registration screen, and taps Sign Up. They enter the name, email, and password. Everything is checked, and a verification code is fired off to the email. The user grabs it from the email and enters it in the app, after which their account is created and the user is directed straight to the login screen.
Alternate Flows	If the email code does not arrive, the user can ask the system to send it again.
Exceptions	if email already exists in the system it shows "Email has already been taken". If the code entered is incorrect, it displays as, "Invalid code, please try again". If the internet is offline, it displays as, "No internet connection".
Includes	MS-002: Email Verification

2.6.2 Select Simulation Scenario

Table 14 Use Case # 2 Select Simulation Scenario

UC Identifier	MS-UC2
UC Name	Select Simulation Scenario
Requirements Traceability	MS-FR-2
Purpose	To let the user choose a specific medical case to practice on.
Priority	High
Preconditions	The user must be logged in and on the main dashboard.
Post Conditions	The chosen scenario is loaded and ready to start.
Actors	Student / Instructor
Extends	No
Main Success Scenario	• User clicks "Start Training" or "Scenarios" button • The system will then show a list of available case scenarios to practice (e.g. Heart Attack, Asthma or ER Emergency). • User reads brief description of the case scenario • User picks the scenario they would like to practice • The system then creates the virtual patient with the correct vitals • User clicks the "Start" button.
Alternate Flows	If the donor wants to edit the product details, they can update the listing before submission.
Exceptions	If product information is not full, system shows: 'All the required fields should be completed.' If image upload failed, system shows: 'Image upload failed! Please try again.'
Includes	None

2.6.3 AI Chat Bot

Table 15 Use Case # 3 AI Chat Bot

UC Identifier	MS-UC3
UC Name	AI Chat Bot
Requirements Traceability	MSFR-3
Purpose	AI Chatbot provides basic guidance over medical queries and support students for learning things related to medical.
Priority	Medium
Preconditions	The user must have logged into the system.
Post Conditions	The student gets the right answers to their queries.
Actors	User
Extends	View chat history
Main Success Scenario	• User open chatbot • Type medical queries • System process queries • Ai generates response displayed to the user
Alternate Flows	If question in unclear or unrelated to medical asks for clarification.
Exceptions	If server unavailable error message is showed. If internet is unstable popup of unstable connection is showed.
Includes	User authentication and NLP module

2.6.4 Generate Performance Report

Table 16 Use Case # 4 Generate Performance Report

UC Identifier	MS-UC4
UC Name	Generate Performance Report
Requirements Traceability	MSFR-4
Purpose	To show the user a summary of how they performed during the simulation.
Priority	Medium
Preconditions	The user must have finished a simulation session.
Post Conditions	A report is created and shown on the screen for the user to read.
Actors	Student / Instructor
Extends	None
Main Success Scenario	The program stops after a set time, or when the user selects to "Finish". The system takes a data file from the session containing all the relevant results such as accuracy and reaction time. The program gives the user a score based on the results of the session. A report page appears displaying the areas the user performed well in, as well as where the errors were made. The user then views the final score or some helpful advice for the next time the program is used. The user then selects Save or Close and returns to the main menu.
Alternate Flows	The user can download the report as a simple file to keep for later.
Exceptions	If the duration of the session is insufficient, the following message is displayed: "Not enough data to create a report". If the saving fails, the following is displayed: "Error saving report. Please check your storage."
Includes	None

2.6.5 Adaptive Quiz System

Table 17 Use Case # 5: Adaptive Quiz System

UC Identifier	MS-UC5
UC Name	Adaptive Quiz System
Requirements Traceability	MSFR-5
Purpose	The Adaptive Quiz System is to assess the student's knowledge dynamically by adjusting question based on their performance.
Priority	High
Preconditions	User must be registered and logged into the MediSim mobile application.
Post Conditions	Quiz results are saved; performance is evaluated.
Actors	Student
Extends	Learning Module
Main Success Scenario	• Student chose adaptive quiz. • System loads first questions. • Student answered the questions. • System evaluates answers in real-time. • Quiz continues based on performance. • Final score and feedback are displayed. • Results are stored in database.
Alternate Flows	If student skips a question, system moves to next question. If incorrect answers increase, system lowers difficulty level. If correct answers increase, system raises difficulty level.
Exceptions	• System error during quiz loading. • Network issue interrupts quiz. • Invalid input or no answer selected
Includes	Question Selection, Answer Evaluation, Score Calculation

2.6.6 Notification System

Table 18 Use Case # 6: Notification System

UC Identifier	MS-UC6
UC Name	Notification system
Requirements Traceability	MSFR-6
Purpose	To provide real time AI based notifications containing important and interesting fact about medical.
Priority	Low
Preconditions	User must be signed-in into the MediSim system.
Post Conditions	User receive accurate and relevant medical facts in the form of notifications.
Actors	Student
Extends	Learning Module
Main Success Scenario	Ai randomly selects the interesting facts and information about the medical field specially EMS facts and information and sends the notification to the user.
Alternate Flows	The AI must attract the user by the unique facts using notifications.
Exceptions	Network connectivity issues prevent notification delivery.
Includes	AI processing module

2.6.7 Skill Scoring and Performance Monitoring

Table 19 Use Case # 7: Skill Scoring and Performance Monitoring

UC Identifier	MS-UC7
UC Name	Skill Scoring and Performance Monitoring
Requirements Traceability	MSFR-7
Purpose	To monitor student performance based on quizzes, learning from daily and simulation activities.
Priority	High
Preconditions	User must be signed-in into the MediSim system
Post Conditions	The system generated and display a performance score and report.
Actors	Student
Extends	Learning Monitoring
Main Success Scenario	• Student completes a simulation. • System records all actions. • Skill scores are calculated. • Performance score is created using Quiz. • Using all performance score a single student report is generated.
Alternate Flows	Student can view past performance and report
Exceptions	If quiz is not recorded proper due to network issue the score cannot be generated.
Includes	Student Performance Monitoring

2.6.8 Session Creating and QR Code and ID

Table 20 Use Case # 8: Session Creating and QR Code

UC Identifier	MS-UC8
UC Name	Session Creation and QR Code and ID
Requirements Traceability	MSFR-8
Purpose	To allow the user to create Session and generate a unique QR code and Id through which students can join that session and learn with the instructor.
Priority	High
Preconditions	User must be signed-in into the MediSim system
Post Conditions	A new simulation session is created and a QR code and id is generated to fast join the session.
Actors	Instructor/Student
Extends	Simulation session
Main Success Scenario	• Creation of a session • Share session via QR code
Alternate Flows	• Instructor can make any user the instructor of the session • If QR code already exists system refresh or regenerate the • QR code.
Exceptions	• Invalid of missing session data. • Database connection failure. • QR code generation Failure.
Includes	Session management and handling

2.6.9 Change Vital Values

Table 21 Use Case # 9: Change Vital Values

UC Identifier	MS-UC9
UC Name	Change Vital Values
Requirements Traceability	MSFR-9
Purpose	To allow the instructor to adjust and control the vital graphs and values during a running simulation.
Priority	High
Preconditions	The instructor is logged in. A simulation session is already running and students are connected to the session
Post Conditions	The updated vital graph and values is shown to all connected students in real time
Actors	Instructor/Student
Extends	Manage Simulations
Main Success Scenario	• Instructor opens the active simulation session. • Instructor selects a vital graph (like ECG or heart rate). • Instructor changes the values using controls/sliders. • System checks the values. • System updates the graph instantly. • Students see the updated graph on their screens.
Alternate Flows	• Instructor can make any user the instructor of the session • If QR code already exists system refresh or regenerate the • QR code.
Exceptions	• Invalid of missing session data • Database connection failure • QR code generation Failure
Includes	Session management and handling

2.7 Summary

How MediSim work and who use it? There are two types of users; instructor which is a full guide and a doctor or student which will use it for learning. The chapter is going to talk about its principal functions; this includes the procedures of log in and starting medical practices and checking patient information. It put attention on making the system efficient, secure and easy for anyone.

Chapter 3: Design and Architecture

3.1 System Design

System design defines the overall structure and behavior of the applications and acts as a blueprint for the development of the project. This section describes how the proposed system is structured, and the different modules of the system interact with each other and how the functional and nonfunctional requirements are translated into a practical solution.

3.2 Design Consideration

Assumptions and Dependencies:

- The user must have basic knowledge of mobile and basic medical education.
- Student will answer quiz honestly without external assistance.
- User will complete quizzes within the given timeframe.

Limitations:

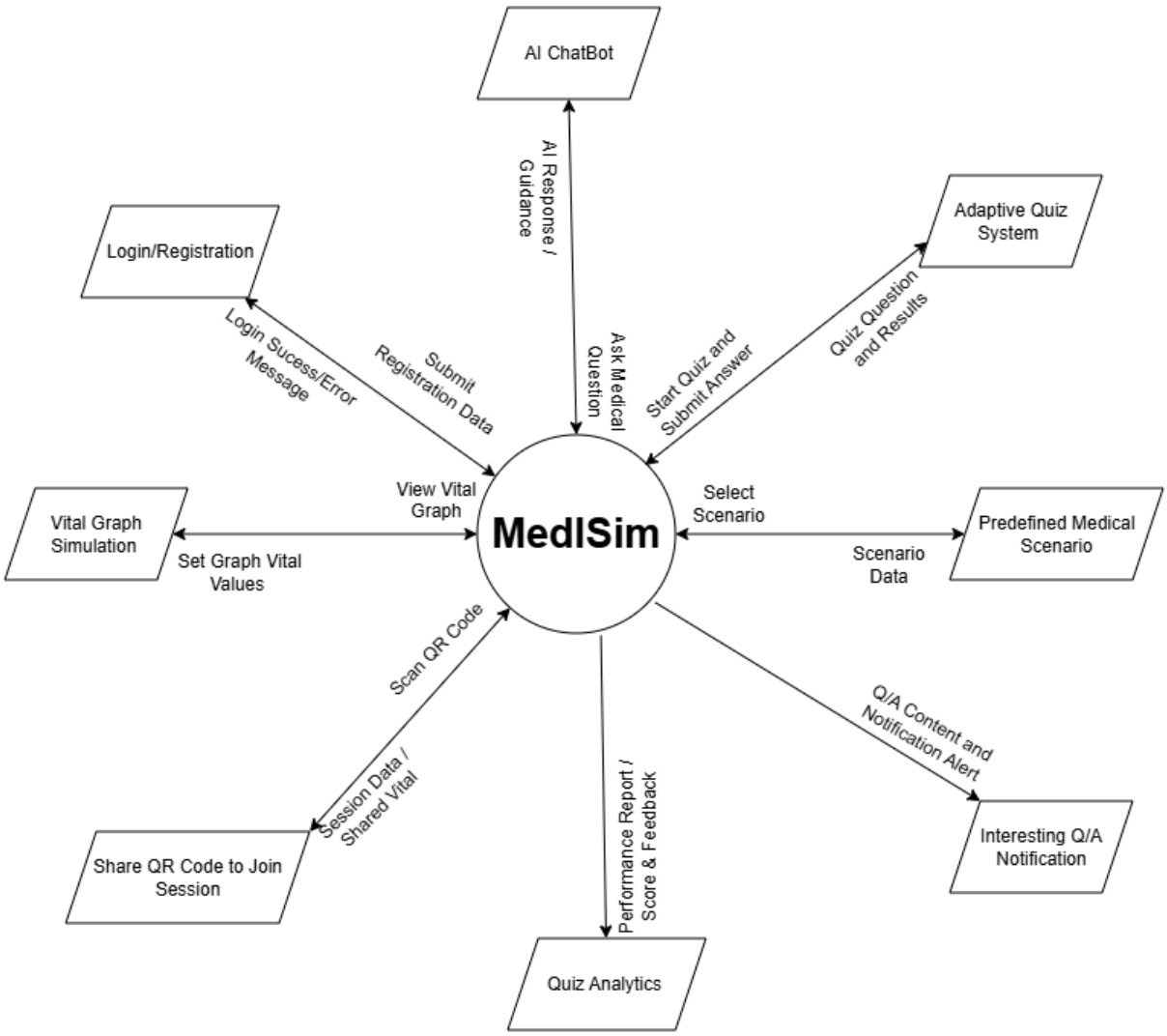
- The system may not fully replicate the real-world scenarios in the App.
- Difficulty adjustment may not always perfectly match student knowledge level.

3.3 Design Model

A design model represents a structured view of the components, relationships, and processes involved in creating the system. The design model views requirements specifications, and use cases for a system, and describes system implementation.

3.3.1 Context Diagram

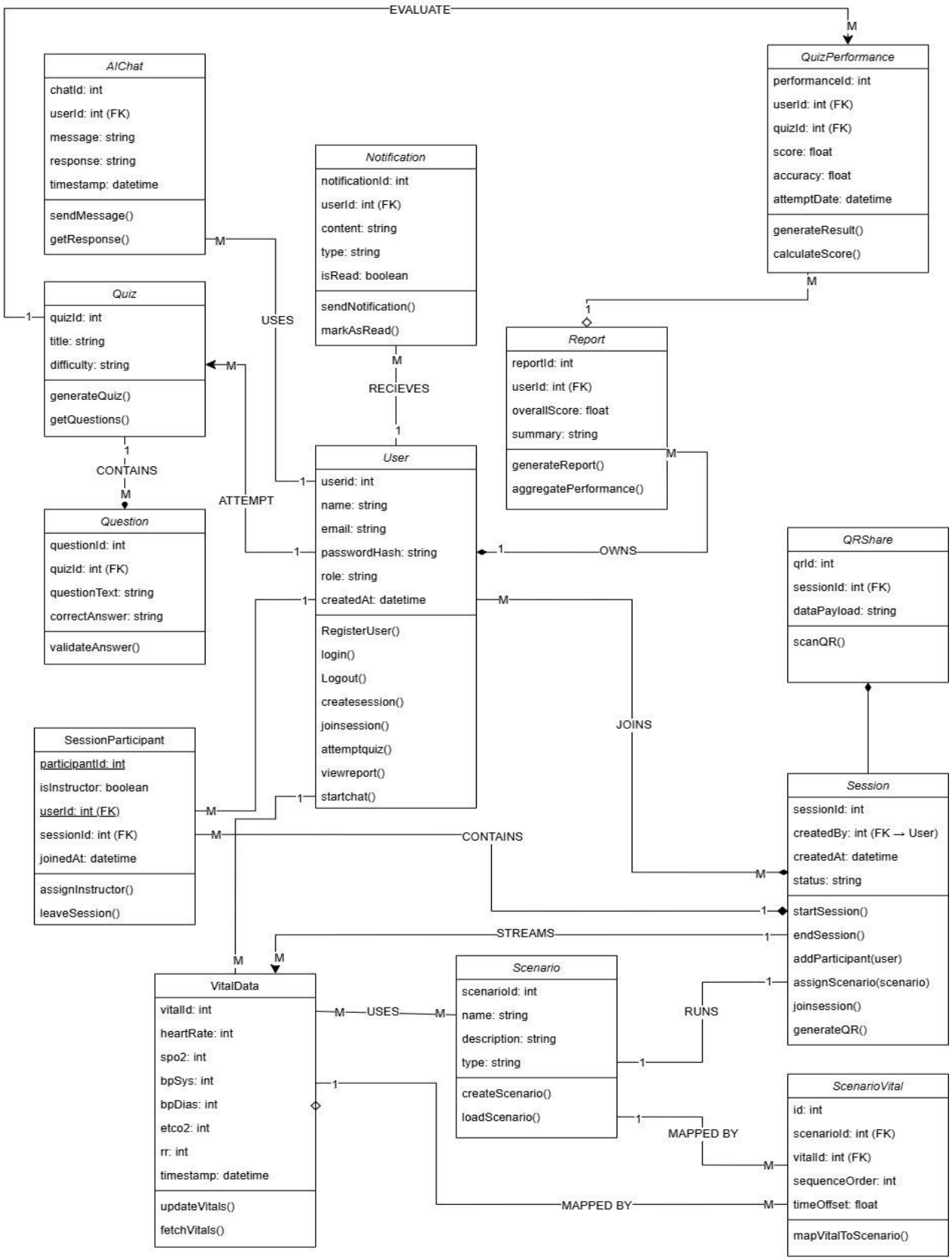
Figure 3: Context Diagram



3.3.2 Design Class Diagram

The class diagram is a structural diagram. The class diagram is used Object Oriented Programming (OOP) technique, where every component represents as a class such as Register User, Admin Management etc. Class diagram describes collection of class, interface, association, aggregation, composition, relationship, attributes and functions. Class diagrams are the most popular UML diagrams used to construct software applications. This class diagram describes the system components as classes and their attributes, operations and the relationship between classes such as the Admin Management class and Register User represent the inheritance relationship. As well as the Register User class represents an association relationship between all the other classes in the system such as Browse Product, Request, Product, Exchange product etc.

Figure 4: Design Class Diagram



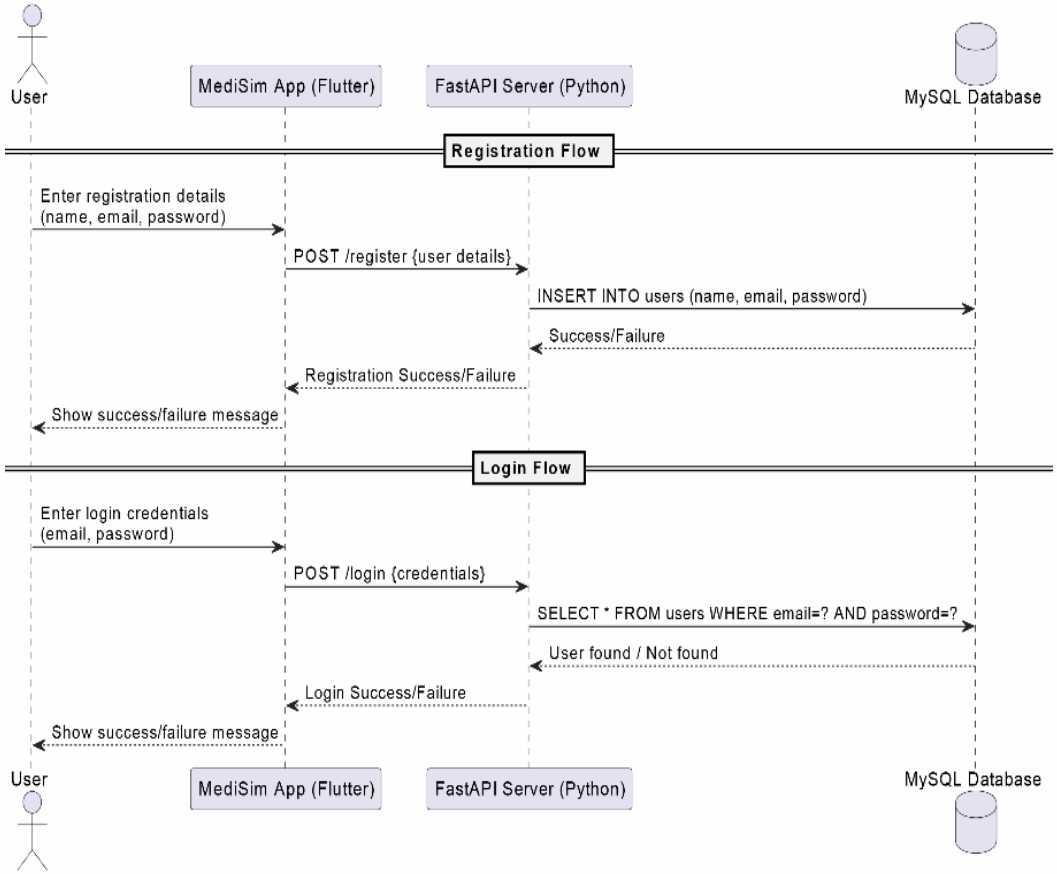
3.3.3 Sequence Diagram

A Sequence diagram is also called an interaction diagram that is used to show the interactive behavior of the system. It demonstrates the interaction of objects in a sequential order. It also provides the order and timing of messages exchanged in a synchronous or asynchronous way. It represents how objects communicate with each other through a series of messages, providing a clear view of the sequence of operations or processes. Sequence diagram notations are lifelines, draw messages, reply/return messages, and activation bars.

- Lifeline shows an object and its participant with timing.
- Draw messages are the messages sent from the sender to the receiver.
- Return messages are the messages sent from the receiver to the sender.
- Activation Bar is added to the object that performs processing or takes action.

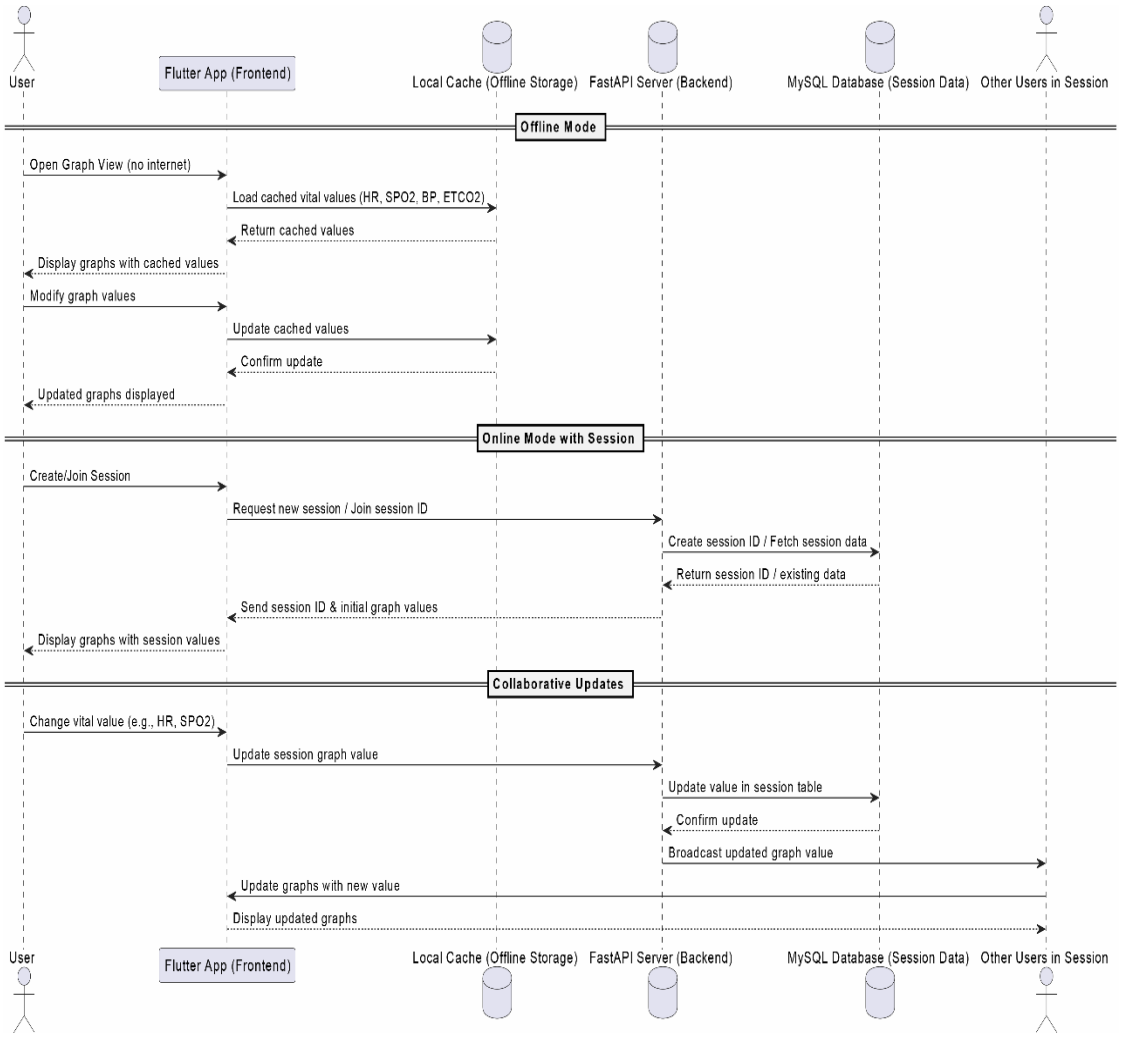
Login and Registration

Figure 5: Sequence Diagram for Login and Registration



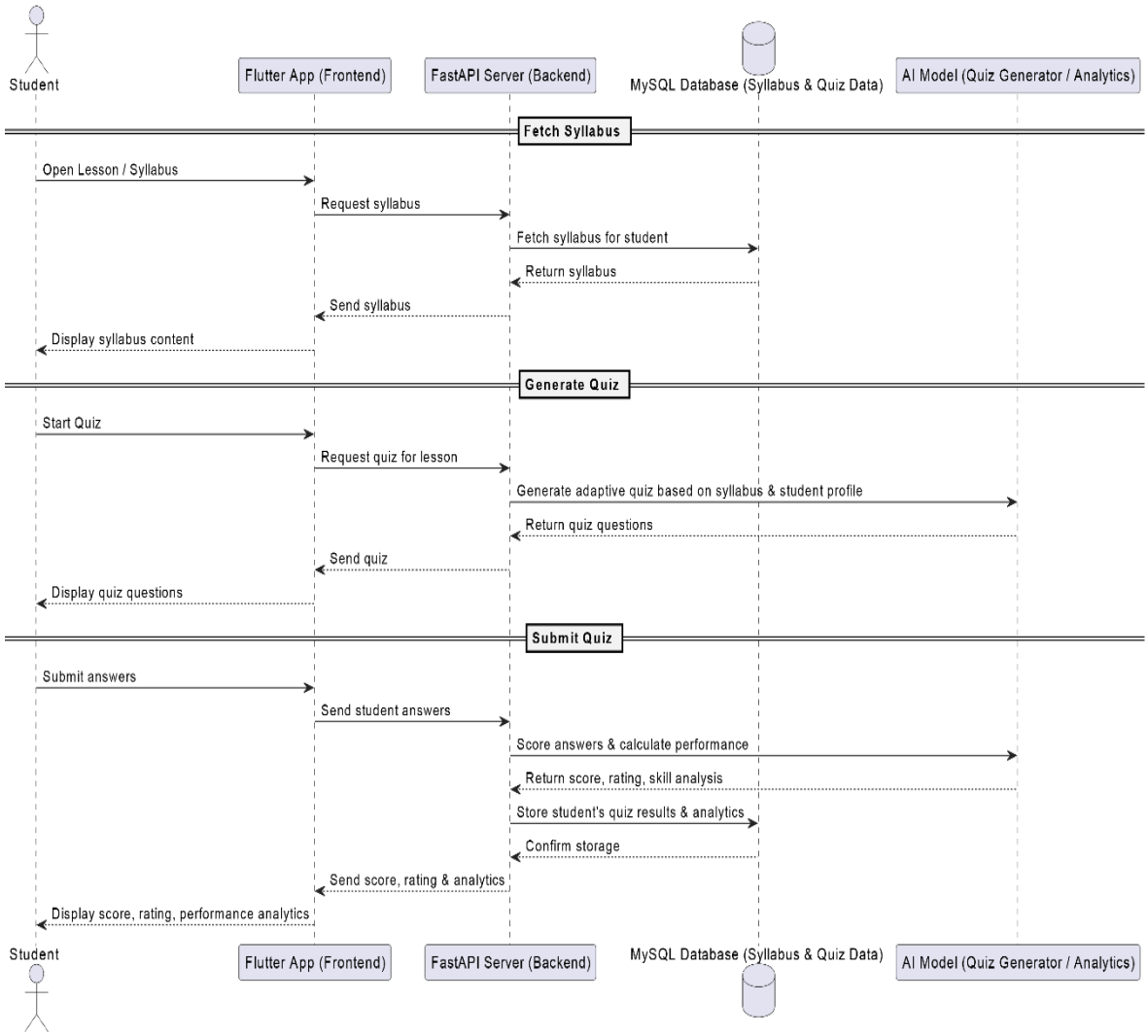
View Graphs

Figure 6: Sequence Diagram for View Graphs



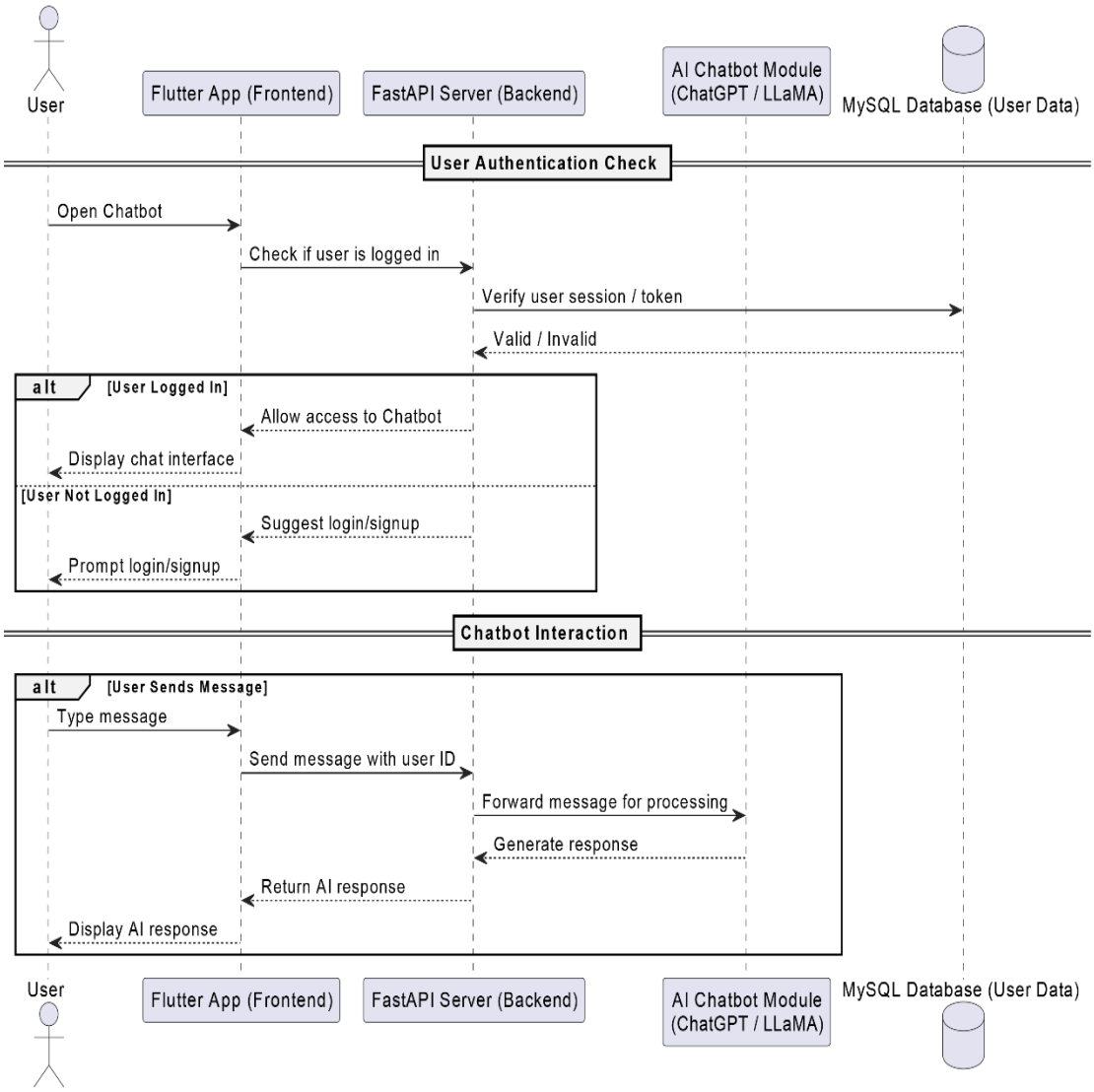
Quiz and Performance

Figure 7: Sequence Diagram for Quiz and Performance



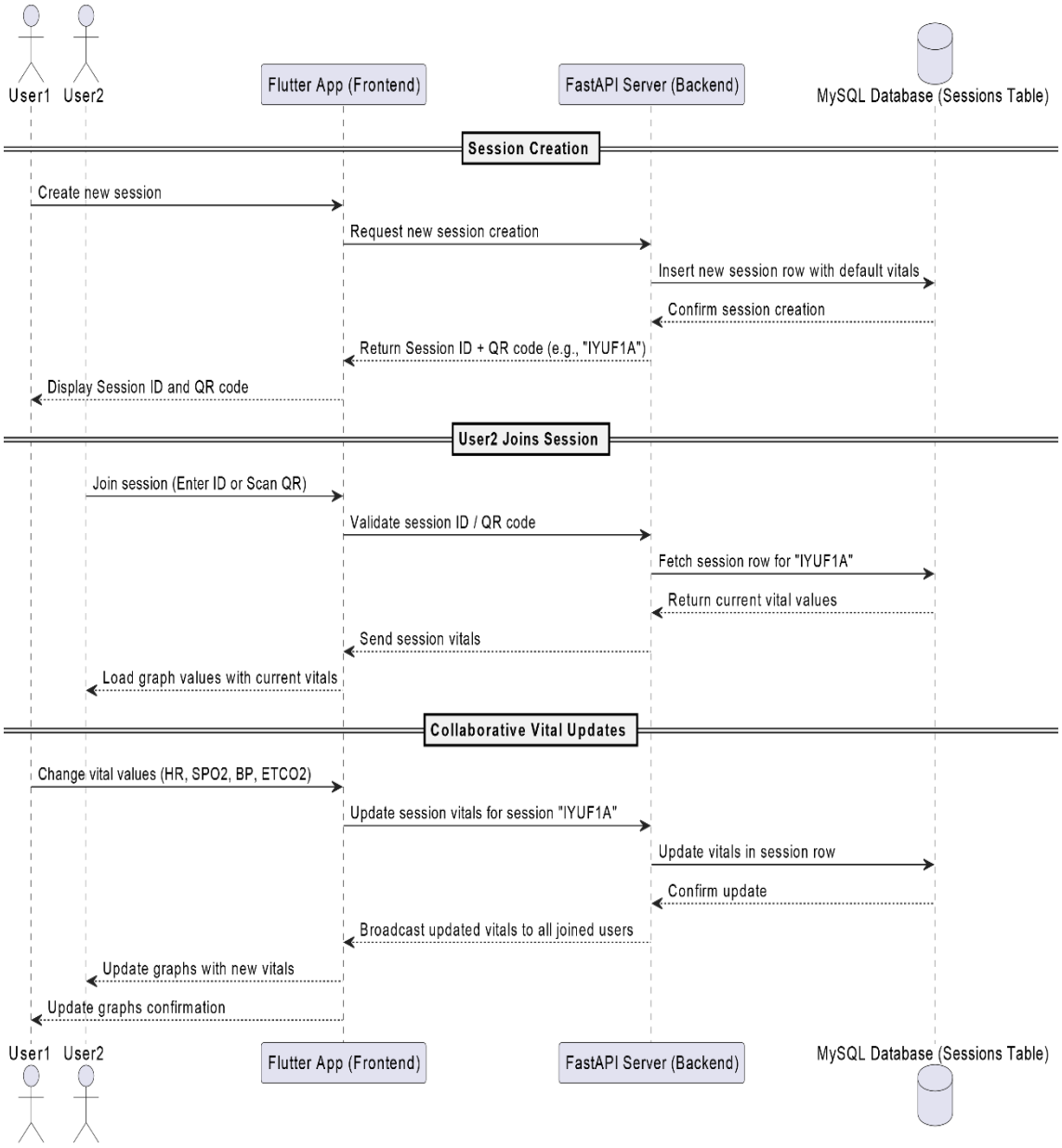
AI Chat Bot

Figure 8: Sequence Diagram for AI Chat Bot



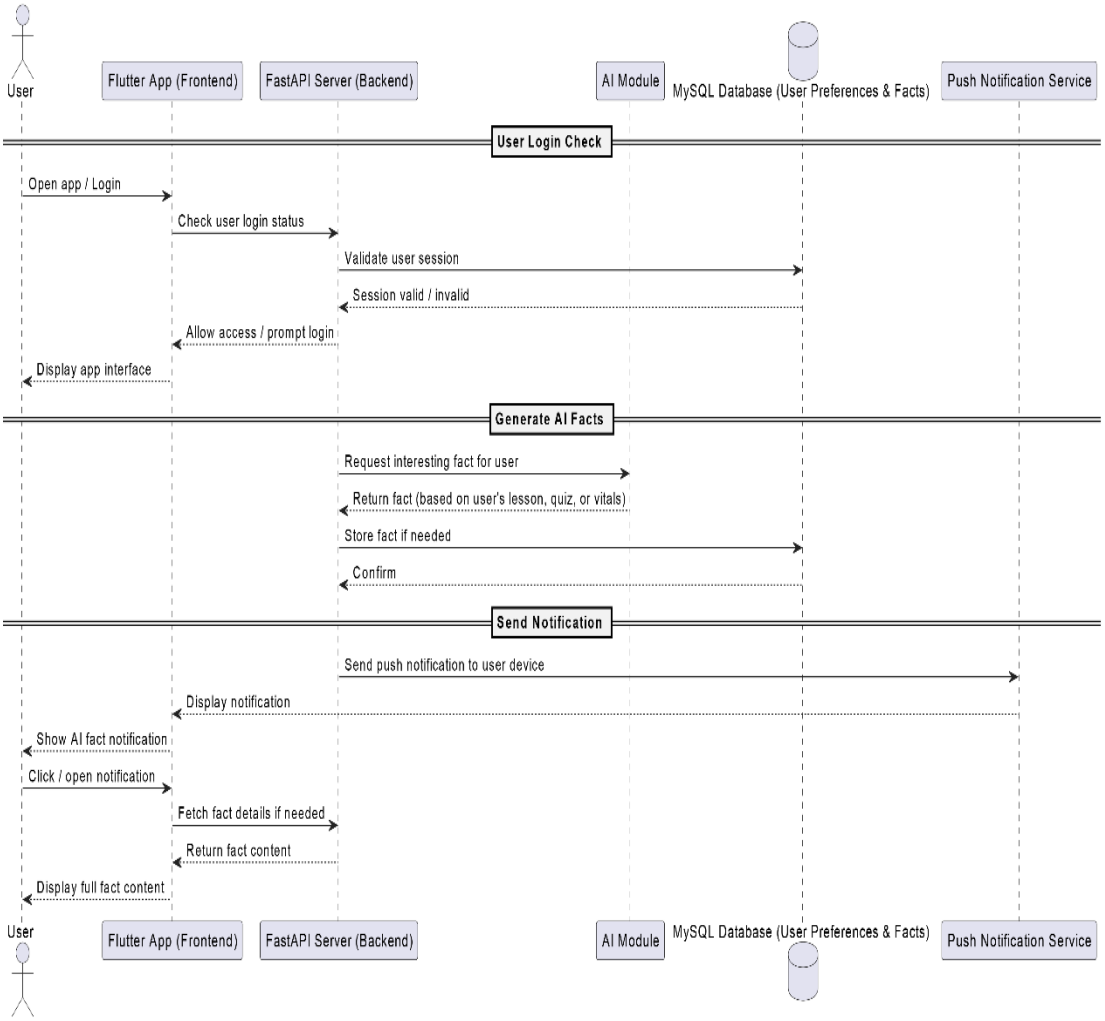
Session ID and QR Code

Figure 9: Sequence Diagram for Session ID and QR Code



Notification System

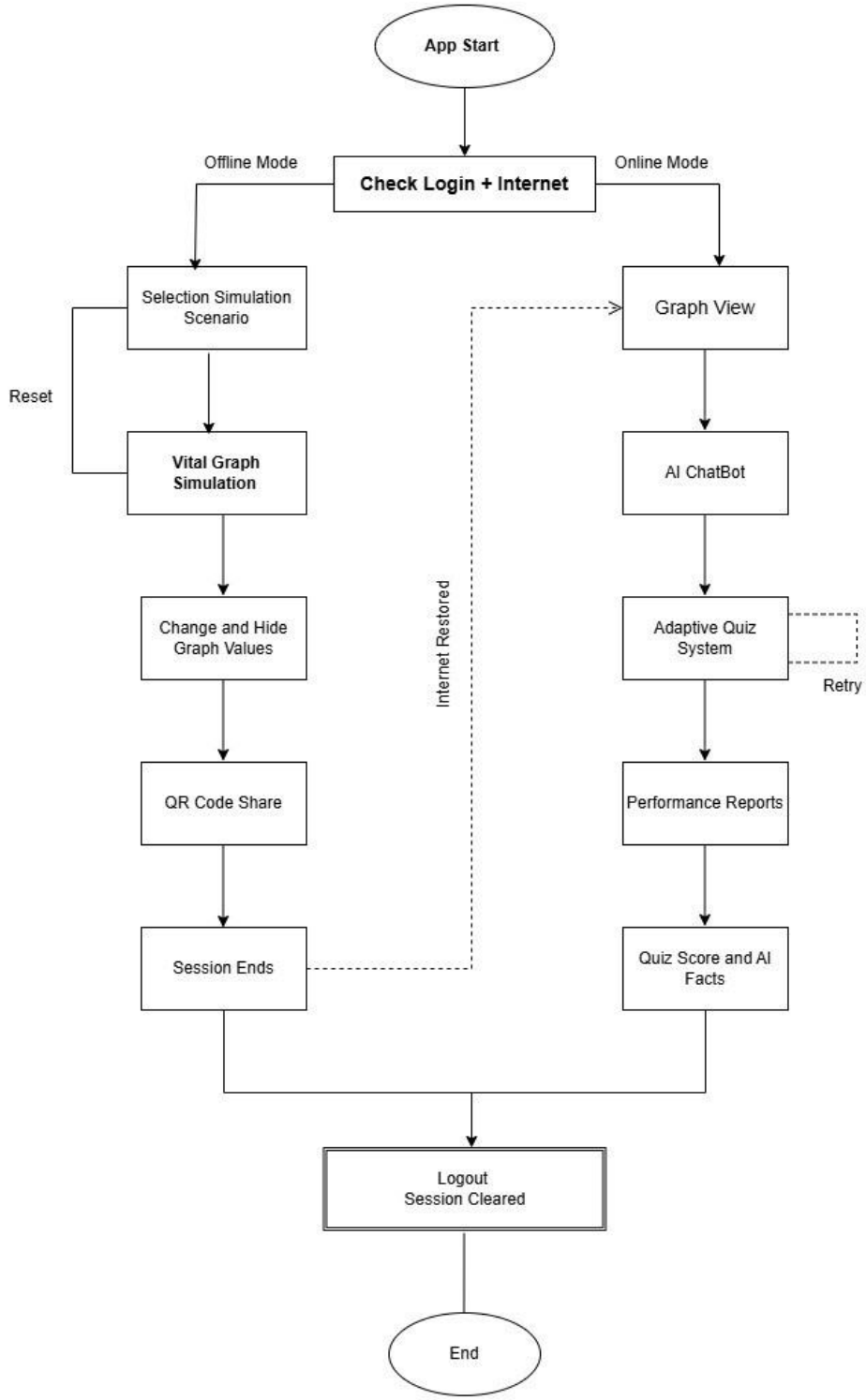
Figure 10: Sequence Diagram for Notification System



3.3.4 State Transition Diagram

The MediSim has a well-defined state transition model where the system initially launches into the activity of graph View state, which is the central hub and main functionality of the project. On the startup of the graph view page, the app checks the login status of the user and internet availability. If the user is not logged in or the internet is not available, the system goes into the offline mode where the graphs are displayed and controlled via cached variables. As soon as the user logs in or the internet is available, the app switches to online mode, enabling all the functionalities of the app.

Figure 11: State Transition Diagram

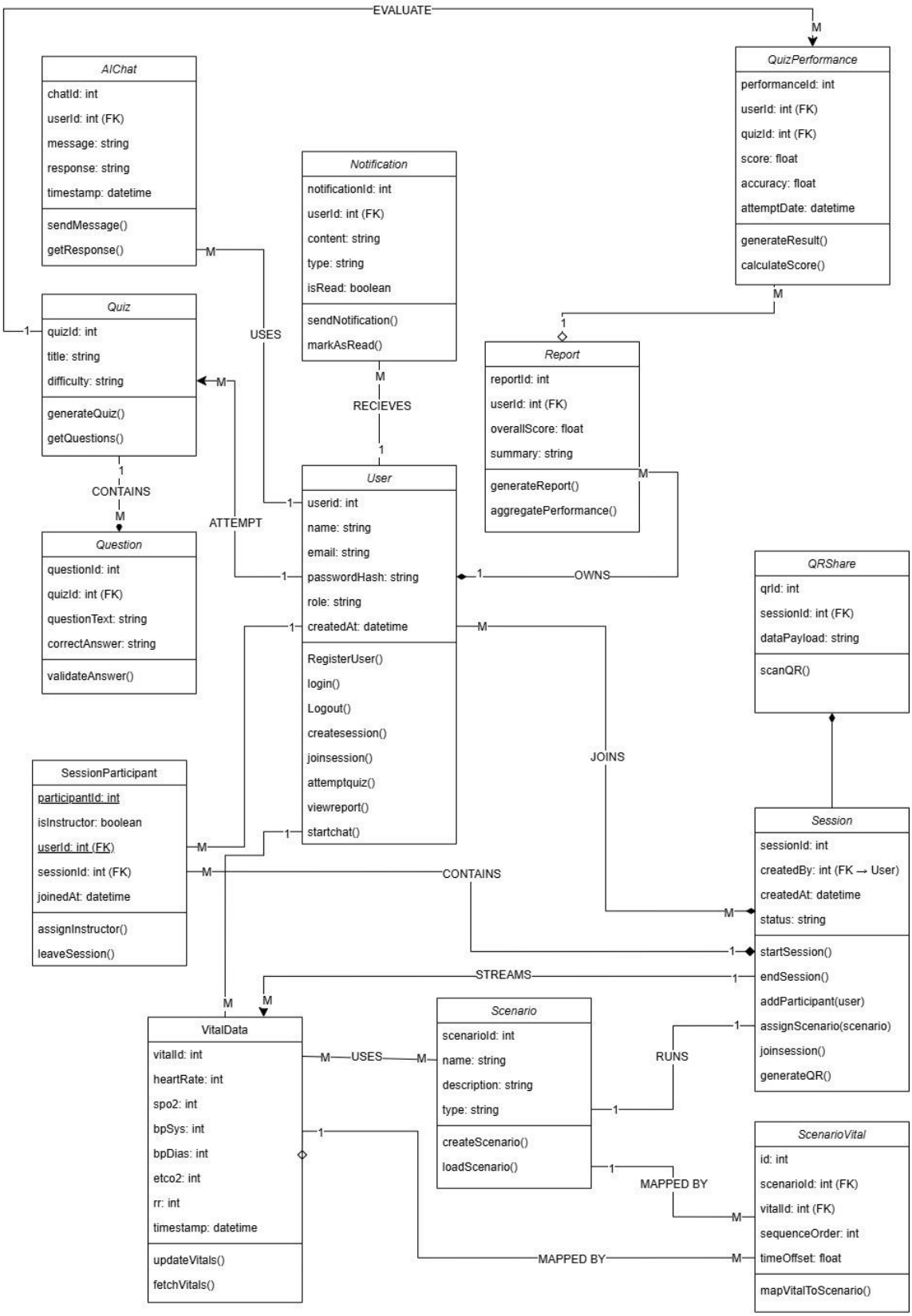


3.4 Architectural Design

Architecture used is client-server architecture where flutter-based frontend acts as a presentation layer that display graphs and maintains the offline cached data. Backend is implemented using Fast API which acts as application and business logic layer, authenticates and manages the session, users and all AI modules, Quiz generation, data synchronization between multiple users.

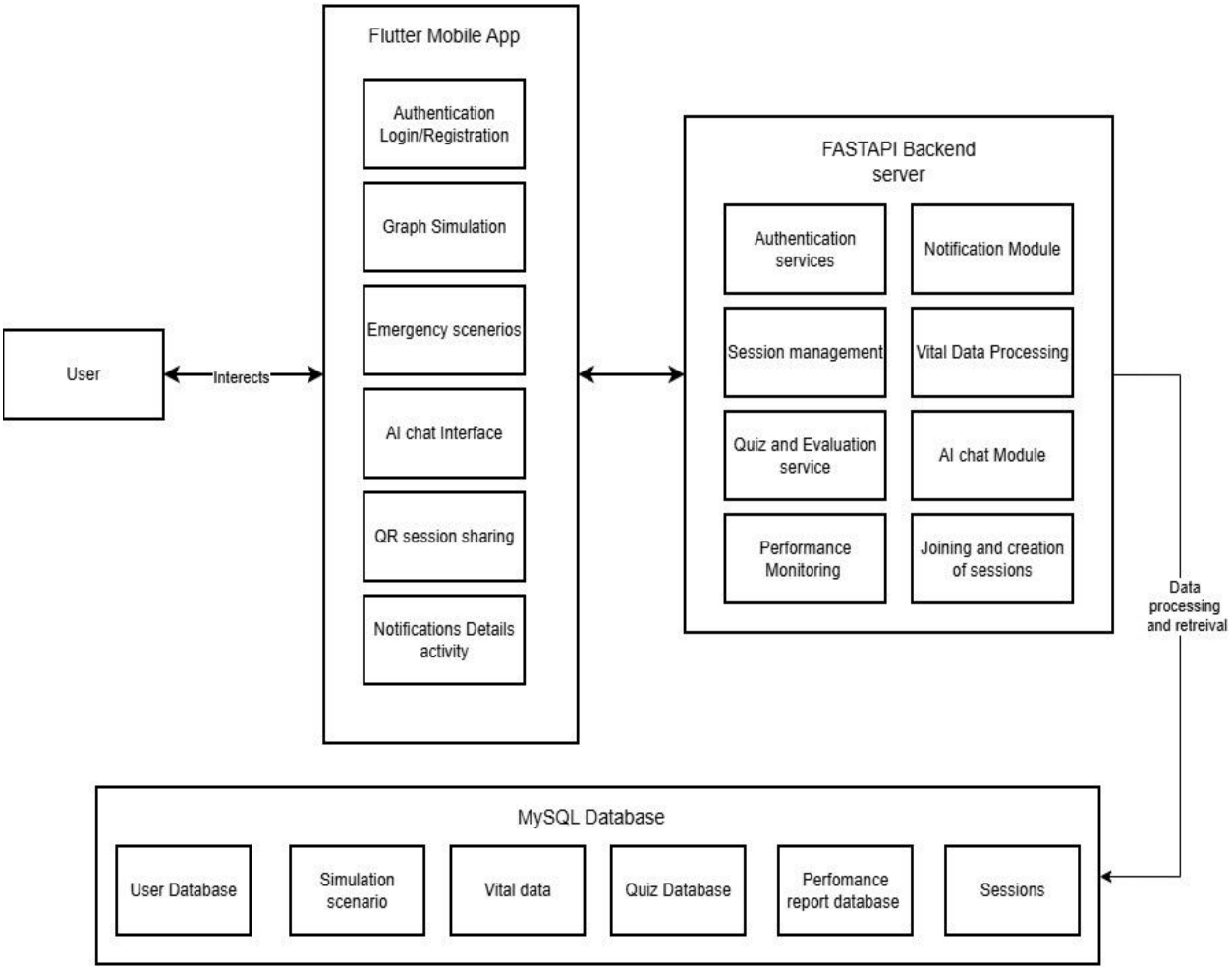
3.4.1 UML Class Relationship Diagram

Figure 12: UML Class Relationship Diagram



3.4.2 UML Component Diagram

Figure 13: UML Component Diagram



3.4.3 Layer Architecture

Table 22 Layer Architecture

Layer	Modules Layer	Responsibilities
Presentation Layer (User Interface)	Frontend (Flutter)	Responsible for handling UI, visualizing graphs and modifying vital values.
Application Layer (Business Logic layer)	Fast API(Python)	Acts as the core of the MediSim Application handling all backend functionalities.
Artificial Intelligence	AI Module	Responsible for handling AI based features of the chat like chat and interesting fact notifications.
Data Layer (Storage)	MySQL	Stores user data, session, vitals, Quiz analytics and AI chats (Optional).

3.5 MVC

Table 23 MVC Component

MVC Component	Modules Assigned
Model (Data & Business Logic)	User data (username, email, password hash), User profile, Session data (Session ID, vital values), Quiz performance records, Cached vital data
View (UI & Notifications)	Flutter UI, Graphs view, Quiz interface, chatbot UI, Session QR and ID, Notifications, control vitals UI
Controller (Handles Requests)	Fast API endpoints, Authentication logic, Session creation & join logic, Vital synchronization logic, Quiz evaluation & analytics logic

3.6 Data Design

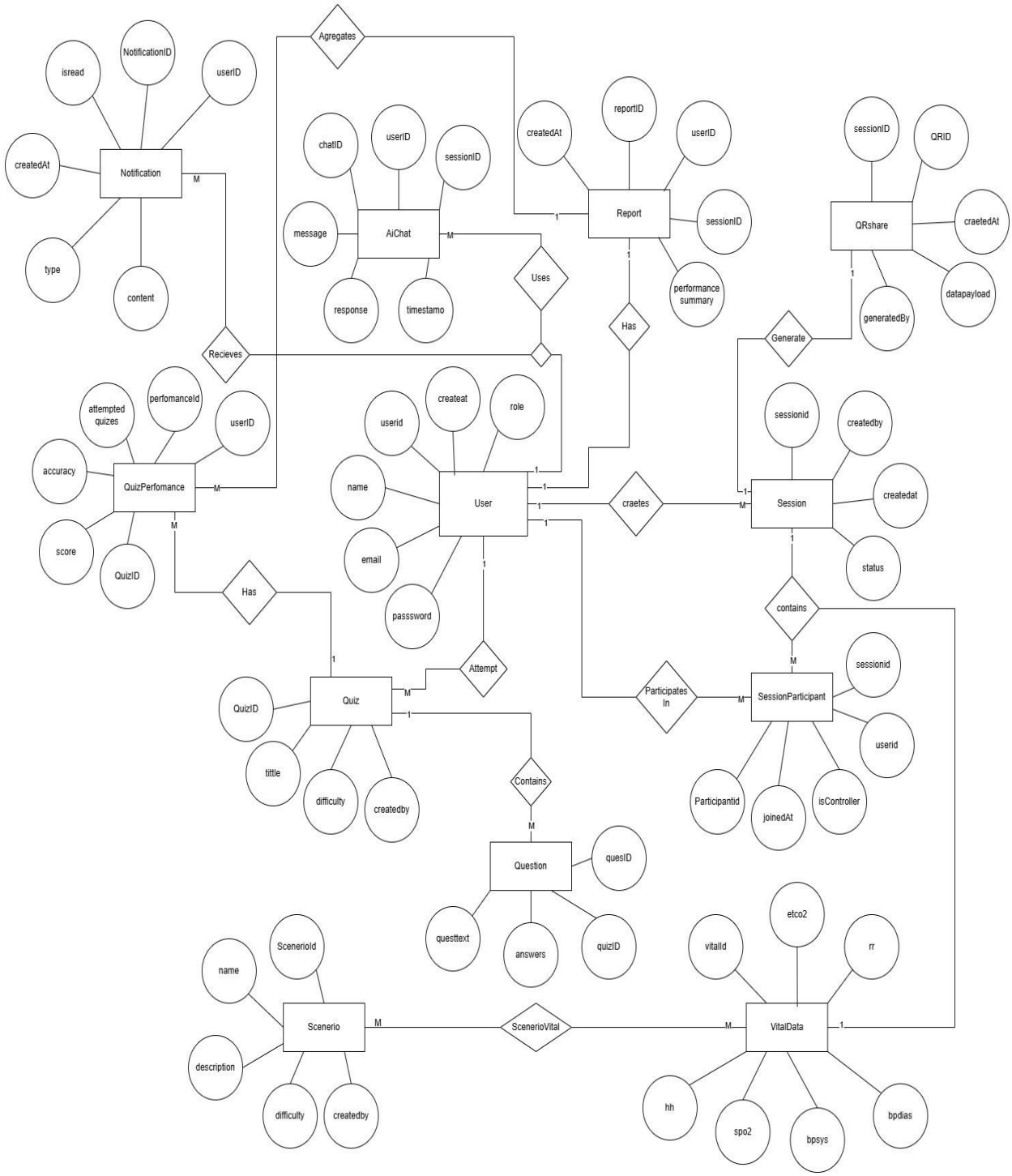
The information is kept in the SQL system: this system records user profile details, session details, quiz results etc. The information is stored within the relationships between user and session details.

Integrity, scalability, efficiency are provided.

3.6.1 ER Diagram

Entity Relationship diagram: This is a model used for identification of which entities is to be stored in database and a description of the relations between entities. An ER diagram has three types of components-Entities, Attributes and relation between entities in a database system. ER diagram is a graphical depiction of entities and their characteristics, displaying how the entities are associated through the relation between entities.

Figure 14: ER Diagram



3.6.2 MYSQL Data Structure

In the MediSim system, the data will be stored, managed and protected using the MySQL. The whole stored data will be stored in tables. It is possible to establish relationship between two tables using foreign and primary key.

The main tables used in MediSim are:

- **User Table**

  This stores details of all system users (students, instructors, etc.) and includes information such as name, email address, password, their associated user role, and account status. This information is essential for logging in and for determining the access rights of individual users.

- **Simulation Table**

  The Simulation Table is a storage for the different scenarios that users can be tested on. Each scenario can be given a title, a type (i.e. Medical, surgical, examination) and a brief description. To prevent invalid entries in this table, the name of the user who created each simulation can be saved here.

- **Result Table**

  This is to store information on particular simulation runs. This will save the details about the performance of a particular user in a simulation, the moves that the user has made during the session and what the results are. This will be important for the storage of a user's progress over time.

- **Vital Sign Table**

  The Vital Sign Table store records patient's vitals, for example, the heart rate, the SpO2 reading, and the patient's blood pressure reading, all of these are stored and kept as values that can be used by the system to demonstrate graphs over time in real time during a simulation run.

- **Notification Table**

  This table stores alert information that will be shown to the user throughout the simulation. For example, there will be a notification to tell the user when any of the vital readings go above an alarm level or whatever other message it would be useful to tell the user at a particular time.

- **Report Table**

  This is to store all generated reports concerning individual simulations. A report would contain general data regarding a patient and the performance of the user on this simulation, graphs etc.,

that will be saved so that the user can later view this data.

3.6.3 Data Processing and Organization

- **Storing Data**
  The MediSim stores all the data that the user entered at the time the user was creating the account or running the simulation. The user's personal details, simulation data and the simulation results are all kept in the database. The database will guarantee the user data to be stored at a secure place where one user cannot enter another user's data.
- **Retrieving Data**
  When a user logged in the system or starting up a simulation, the system obtains the required data from the database, for example; the system is obtaining the simulation that running details, the historical simulation result of the simulation that running and so on and the vital signs parameter. By retrieve the data the system performance not dropped at all.
- **Processing Data**
  MediSim's function in this area is to take all the inputs and process them and to present the result to the user in real-time. For example if the simulation was run it would continuously update and show the value of the vital sign along with the information presented in a graph, the system would check if the reading was out of the normal range and alarm the user, it would also use saved information to create reports and to aid the user.

3.6.4 Data Dictionary

Table 24 Data Dictionary

Entity / Attribute / Method	**Description**
UserID	Unique ID of the user.
Username	User’s display name.
Email	Email of the user.
Password hash	Encrypted password of the user

Role	Role of user in the system
CreateDate	Date/time when the user is created
PerformanceID	Unique key for the Quiz performance
UserID	User who achieved the performance
QuizID	Unique ID for the Quiz
Accuracy	Accuracy of quiz
Score	Score in the quiz
AttemptDate	Date quiz attempted
VitalID	Vital unique id
Timetsamp	Date time for vital creation
HR	Heart rate of the graph
SPO2	SPO2 % of the graph
BPsys	Systolic BP of the graph
BPdias	Diastolic Bp of the graph
ETCO2	ETCO2 of the graph
RR	Respiratory Rate of the graph
SessionID	Unique id for session.
Createdby	Who created session ID
CreateAT	Time of creation
Status	Status of session whether it's live or not

ChatID	Unique ID for chat
Message	Message of chat
Response	Response of AI in chat
QuizTitle	Title for quiz
Difficulty	Difficulty level for quiz
NotificationID	Unique ID for notification
Content	Notification content
Type	Notification type (fact, quiz, reminder)
Isread	Bool for notification read or not
ReportID	Unique ID for report
OverallScore	Overall score of students in report
Summary	Summary of student in report
QuestionID	Unique ID of question
QuestionTxt	Actual question stored in variable form
CorrectAns	Correct answer in system
QRID	Unique id for QR code
Datapayload	Data stored in QR
SessionParticipant	Participant in a session
IsInstructor	Boolean whether the user is instructor
JoinedAt	Time when user joined session

ScenarioID	Unique ID for scenario
Name	Scenario name
Description	Description of the scenario
Type	Type of the scenario
VitalID	Id of individual vitals
SequenceOrder	Order of sequence of the scenario

3.6.5 Functions and Parameter

Table 25 Functions and Parameter

RegisterUser()	Name, email, password, phone	User registration
Login()	Email, password	Login user
Logout()	UserID	Logout user
CreateSession()	UserID	Create a new session
JoinSession()	Session ID, User ID	Join an existing session
ViewReport()	UserID	View a student report
AttempQuiz()	UserID, QuizID	Attempt a quiz
StartChat()	UserID	Start an AI chat
Sendmsg()	UserID, Message	Send message in AI chat
GetResponse()	ChatID	Get response from AI chat
GenerateQuiz()	Topic	Generate Quiz from Syllabus
Getques()	QuizID, Queslist	Get generate question from quiz
ValidateAns()	QuesID, SelectedAns	Validate answers from quiz
AssignInsturct()	ParticipantID	Assign a user as session instructor
LeaveSession()	ParticipantID	Leave a joined session

Updatevitals()	SessionID, VitalData	Update vital values
Fetchvitals()	SessionID, List	Fetch vitals
Sendnotification()	UserID, Content	Send notification to user
Markasread()	NotificationID	Mrak notification as read
Createscenario()	Name, type CreatedBy	Create patient scenario
Loadscenario()	ScenarioID	Load patient scenario
Generatereport()	UserID, Performance	Generate student report
AggregatePerformance()	UserID	Aggregate student performance into report
GenerateResult()	PerfomanceID	Generate quiz performance report
ScanQR()	QRData	QR scanner
StartSession()	SessionID	Start a new session
EndSession()	SessionID	End the session

3.6.6 Object Oriented Approach

1. Object: User

Attributes

- user-Id: int
- name: string
- email: string
- password: string
- createdat: datetime

Methods

- register ()
- login ()
- logout ()

- createSession()
- joinSession()
- attemptQuiz()
- viewReport()
- useAIChat()

2 Object: Session

Attributes

- sessionid: int
- createdAt: datetime
- status: string

Methods

- startSession()
- endSession()
- endSession()
- generateQR()

3 Object: Session Participant

Attributes

- IsController: boolean
- JoinedAt: Datetime
- isInstructor: boolean

Methods

- assignInstructor()
- endSession()

4 Object: Quiz Performance

Attributes

- performance-Id: int
- score: float
- accuracy: float
- attemptdate: datetime

Methods

- checkperfomace()

5 Object: Vital Data

Attributes

- VitalID: Int
- heartrate: int
- spo2: int
- bpsys: int
- bpdias: int
- etco2: int
- rr: int

Methods

- updatevitals ()
- fetchvitals()

6 Object: Scenarios

Attributes

- scenario-Id: int
- name: string
- type: string

Methods

- Createscenerio()
- loadScenerio()

7 Object: Quiz

Attributes

- Quiz-Id: int
- title: string
- difficulty: string

Methods

- generatequiz()

8 Object: Question

Attributes

- question-Id: int

- questiontxt: string
- options: string
- correctans: string

Methods

- validateans()

9 Object: Report

Attributes

- report-Id: int
- overallscore: float
- summary: string

Methods

- generatereport()
- updatereport()

10 Object: AI Chat

Attributes

- chat-Id: int
- message: string
- response: string
- timestamp: date/time

Methods

- sendmsg()
- recieveresp()

11 Object: Notification

Attributes

- notify-Id: int
- content: string
- type: string
- createdAt: date/time
- isread: Bool

Methods

- sendnotify()
- markasread()

12 Object: QR code Share

Attributes

- qr-Id: int
- datapayload: string
- createdAt: date/time

Methods

- generateQR()
- scanQR()

3.7 User Interface Design

The user interface is configurable and easy to navigate as well as being user-friendly. The Graphs are smooth and resemble real time graphs. The user is provided easy access to the features and requirements needed for using system.

3.7.1 Screen Images

Figure 15: Login Screen and Share QR Code

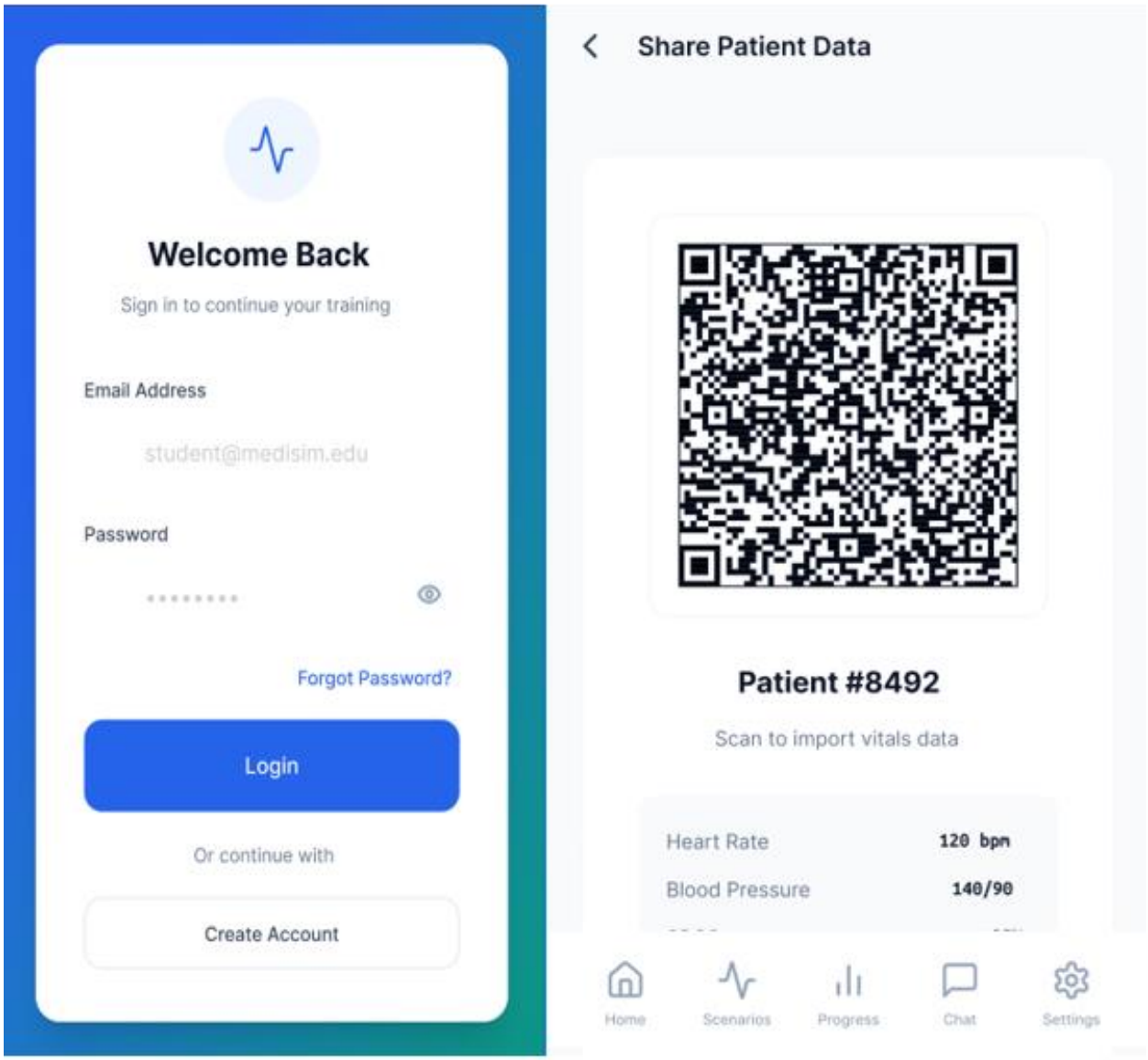


Figure 16: View Graphs and Scenarios

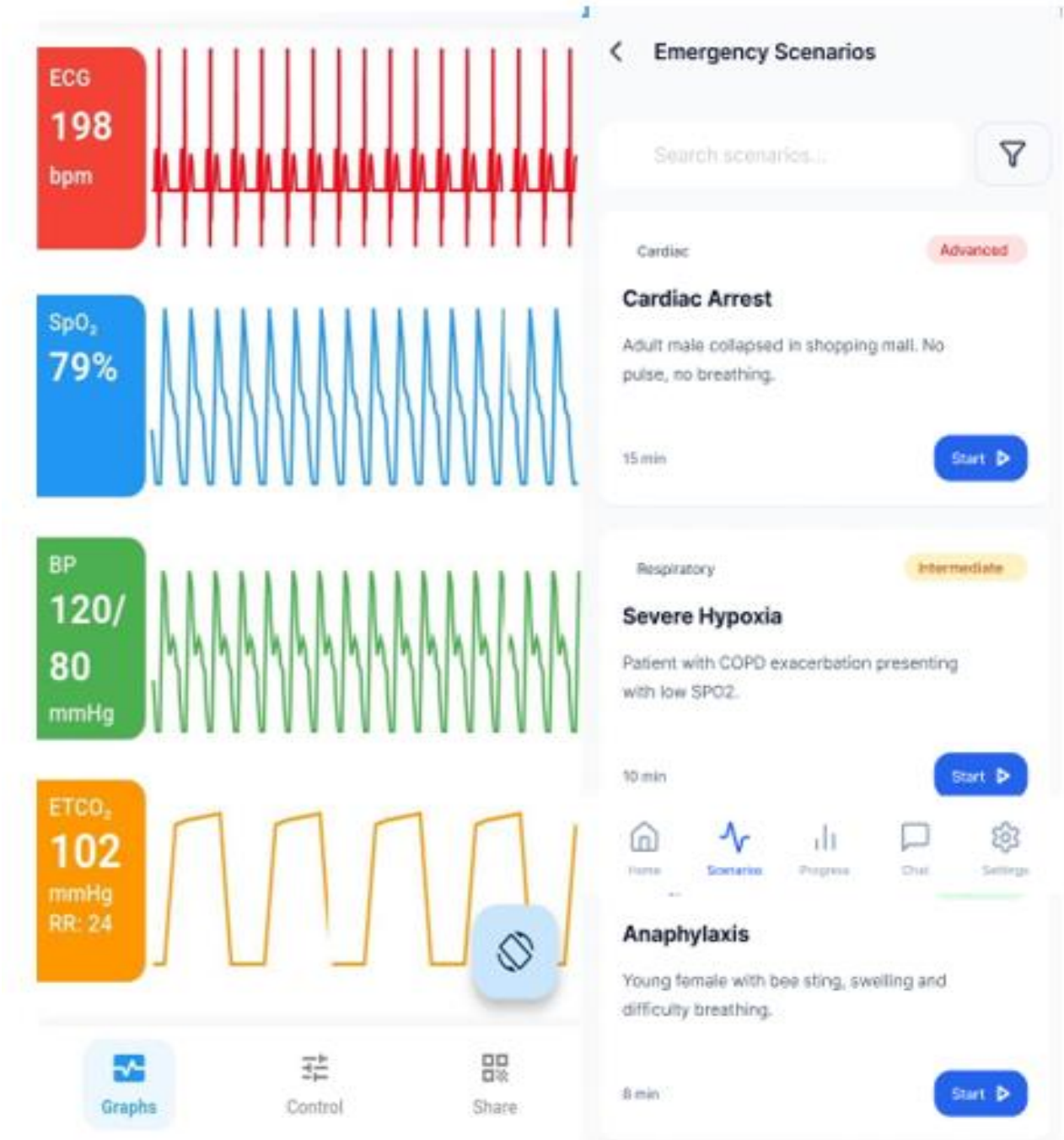
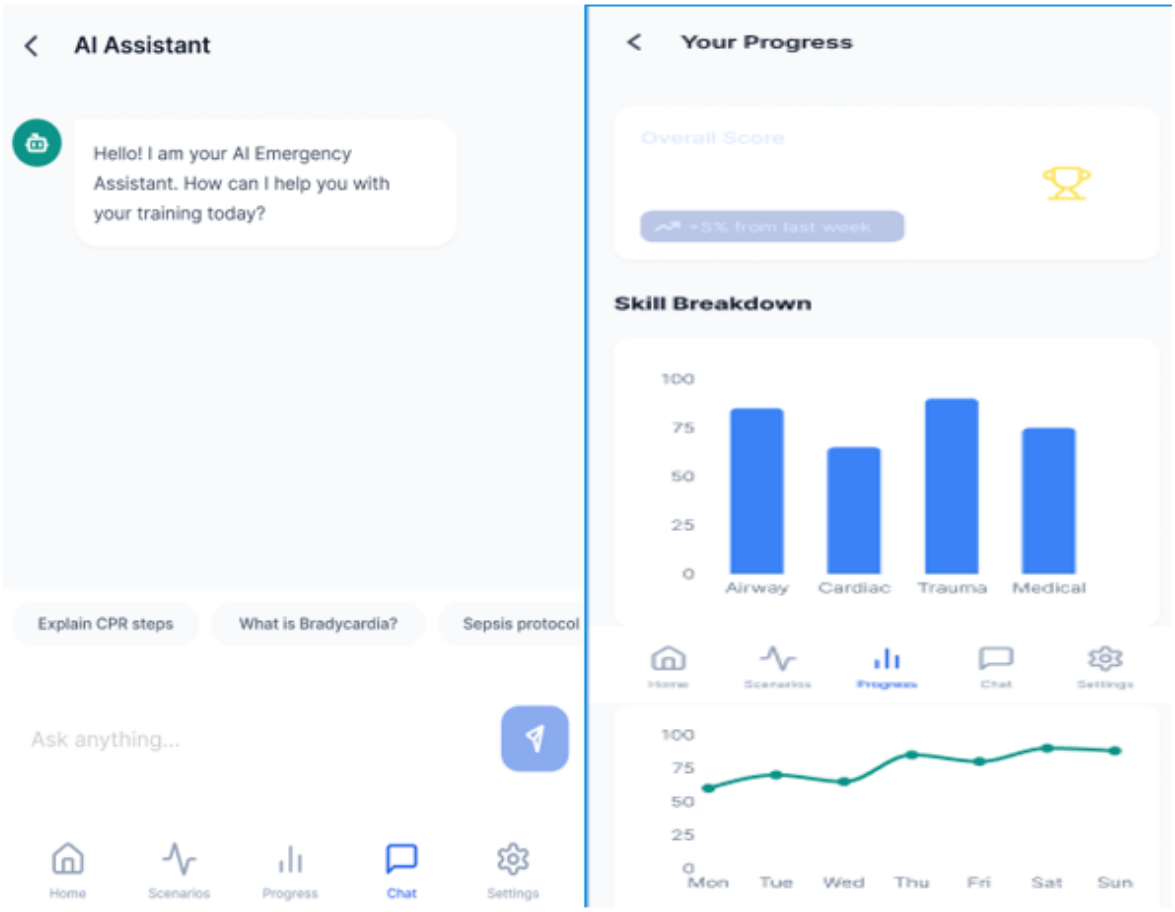
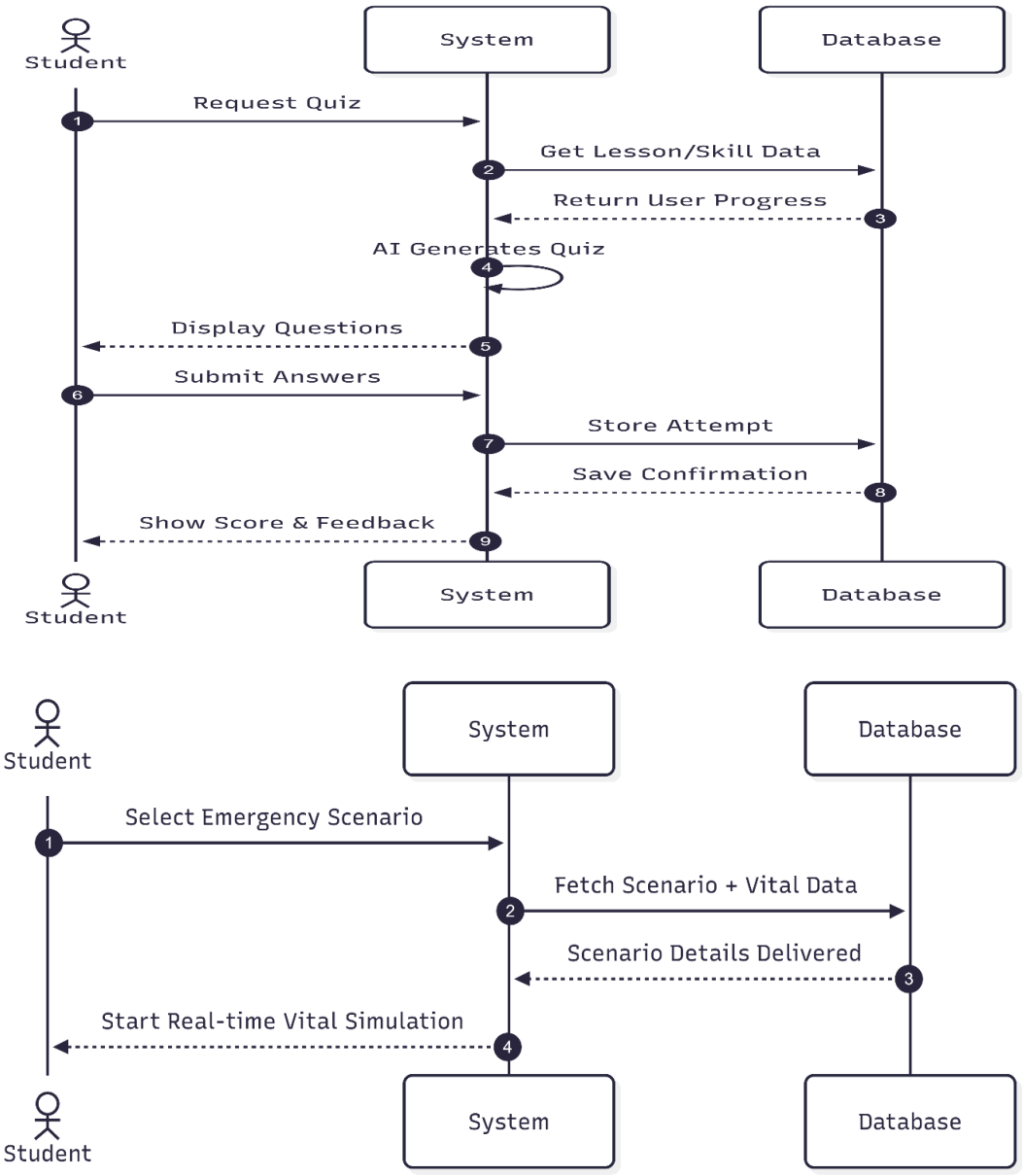


Figure 17: AI Assistant and Progress Screen



3.8 Behavioral Model

Figure 18: Behavioral Model



3.9 Design Decision

The design decisions for MediSim made to ensure the system is efficient, user-friendly, and scalable. These decisions are based on the needs of users and instructor as well as the challenges faced by traditional books and unstructured graphs in different app.

System Architecture Design

- **Choice:** Layered architecture with MVC pattern
- **Reason:** This design principles used within MediSim contribute to keeping the overall design simple and manageable. The user interfaces (i.e. Graphs, dashboards) and the logic component have been separated from the database components. This allows for quick and independent changes and updates to be made to the database or logic design without influencing other system components. The ease of development can also be attributed to separation of components as individual units can be assigned to be developed simultaneously by different teams.

Data Design

- **Choice:** Relational database using MySQL
- **Reason:** The choice for using MySQL was due to it being simple, reliable and easily manageable. MySQL is utilized as it is a database that stores information into tables (such as user tables, simulation tables and results tables). Interlinked tables are crucial as it keeps the database properly connected which is vital for MediSim as most components rely upon linked and correct data.

Object-Oriented Design Pattern

- **Choice:** Singleton and Factory design patterns
- **Reason:** Singleton pattern used to prevent creation of a significant service like database connection, which used by entire project. It prevents to cause confusion in project, to use resource like memory. Factory pattern allows the creation of object like simulation types and reports without giving an additional work. It creates the objects very clean.

Simulation & Logic Design

- **Choice:** Real-time data handling and event-based logic
- **Reason:** Since the input data for the MediSim system are live data (heart rate, oxygen

level) the system must react dynamically when a value changes. If a certain vital sign has a value higher or lower than acceptable, the MediSim system will display a warning.

Security & Authentication

- **Choice:** Role-Based Access Control (RBAC)
- **Reason:** Different users have different roles like student and instructor. RBAC makes sure each user can only access what they are allowed to see. It also keeps user data safe and protects the system from unauthorized access.

Notifications & Alerts

- **Choice:** In-system alerts and notifications
- **Reason:** MediSim gives alerts during simulation when something important happens, like abnormal vital signs. These notifications help users react quickly and learn better. It also improves the overall experience.

User Interface Design

- **Choice:** Simple and responsive UI
- **Reason:** The interface is designed to be clean and easy to use. Students can easily understand graphs, values, and controls.

3.10 Summary

Our system follows a multi-layered MVC design for separating backend, frontend and data layer. MySQL is used to handle data efficiently as it is capable of storing large amounts of data effectively. Also, our system follows modular design which increase flexibility and maintainability of our system. The system includes a login/registration page where only registered user can use online features. Flutter is used to design and show efficient and user-friendly graphs for students.

Chapter 4: Implementation

4.1 Implementation

Implementation is the process of converting a software design into a functional software system. It is the stage where the detailed design is translated into code and the software is built, integrated and tested to ensure that it meets the already specified requirements in the SRS. In this phase, computer code is written and executed to test and deploy the application. This chapter documented the steps and techniques used to implement the system design.

4.1.1 Algorithm

An algorithm is a precise set of steps or guidance that can be used to solve a problem or be used in doing a specific task. It is the actual stage where the real implementation of the proposed and designed solutions to complex problems are being developed with efficiency and effectiveness. They allow processes to be automated which translates to reliability, speed and ease of execution.

Algorithm: Login/Registration

Table 26 Login/Registration

Algorithm 1: Login/Registration
Input: Username, Email, Password
Output: Account created successfully or error message
Step 1: Registration Process User enters Username, Email, and Password Check if Email already exists in database If Email already exists, return "Account Already Exists" Else: Save user data in database Return: "Account Created Successfully" **Step 2: Login Process** User enters Email and Password Retrieve user data from database using Email If Email not found, return "User Not Registered" Check if Password is correct If Password is correct: Start user session Else: Return "Invalid Password" **End**

Algorithm: Vital Graph Simulation

Table 27 Vital Graph Simulation

Algorithm 2: Vital Graph Simulation
Input: Simulation Data
Output: Live Graph and updated vital values
Step 1: Start Simulation User selects a simulation scenario System loads related vital data **Step 2: Initial Graph** Graph area is displayed on screen Initial values of vital signs are shown **Step 3: Updated Values** System updates heart rate, SpO2, and BP New values are taken from simulation data **Step 4: Display Graph** Graph lines move smoothly with new values Old values shift left and new values on right **Step 5: Continue Simulation** Graph keeps updating again and again Values change until simulation ends **Step 6: End Simulation** Stop updating values Show final graph and results **End**

Algorithm: Adaptive Quiz System

Table 28 Adaptive Quiz System

Algorithm 3: Adaptive Quiz System
Input: User answer, Question bank
Output: Next Question / Final Result
Step 1: Start Quiz User start the quiz System shows first question **Step 2: User Answer** User selects an answer System checks if answer is correct or wrong **Step 3: Continue Quiz** Repeat question and answer process keeps changing based on user answers **Step 4: End Quiz** Quiz ends after all questions System calculates score **Step 5: Show Result** Display final score Show performance level (good / average / weak) **End**

Algorithm: AI Chat Bot

Table 29 AI Chat Bot

Algorithm 4: AI Chat Bot
Input: User Question, Message
Output: Chat Bot reply, answer
Step 1: Start Chat User opens chatbot User types a question or message **Step 2: Receive Input** System receives user message Message is prepared for processing **Step 3: Understand Message** System checks the message It tries to understand what user is asking **Step 4: Find Answer** System searches for related answer from stored data Match the question with available information **Step 5: Generate Reply** System creates a simple and clear answer Prepare response for user **Step 6: Show Response** Chatbot sends reply to user Answer is shown on screen **Step 7: Continue Chat** User can ask another question Process repeats again **End**

Algorithm: Share Vital via QR Code and ID

Table 30 Share Vital via QR Code and ID

Algorithm 5: Share Vital via QR Code and ID
Input: Vital Data, User ID
Output: QR Code generated / ID shared
Step 1: Select Share Option User clicks on "Share Vital" button System shows options: QR Code or ID **Step 2: Generate Share Data** System collects current vital data Create a unique ID for this data **Step 3: QR Code Generation** If user selects QR Code System creates QR code using the ID QR code is shown on screen **Step 4: Share via ID** If user selects ID option System shows the unique ID User can copy and share this ID **Step 5: Access Data** Other user scans QR code or enters ID System finds the data using ID Show vital data on screen **Step 6: Validation** Check if ID is valid If ID not found, show "Invalid ID" **End**

Algorithm: Skill Scoring and AI Generated Reports

Table 31 Skill Scoring and AI Generated Reports

Algorithm 6: Skill Scoring and AI Generated Reports
Input: User Skills, Test Answers, Activity Data
Output: Skill Score and Report
Step 1: Start Process User opens the skill test or task System shows questions or tasks **Step 2: Collect Data** User answers questions or completes tasks System saves all answers and activity **Step 3: Check Answers** System checks correct and wrong answers System gives marks for each answer **Step 4: Calculate Score** System adds all marks System creates a total skill score **Step 5: Analyze Skills** System finds strong and weak areas System understands user performance **Step 6: Generate Report** System creates a simple report Report shows score, strengths, and weak points **Step 7: Show Report** User can see the report on screen User can download or share the report **Step 8: Validation** Check if all data is correct If error, show “Data Error” **End**

Algorithm: Interesting Notification

Table 32 Interesting Notification

Algorithm 7: Interesting Notification
Input: User activity, App data
Output: Notification shown to user
Step 1: Collect Data System checks user activity System looks at recent actions **Step 2: Find Interest** System finds what user likes Example: games, study, news **Step 3: Create Notification** System makes a short message Message is simple and clear **Step 4: Show Notification** Notification appears on screen User can tap to open **Step 5: User Action** User reads or ignores it If tapped, app opens **Step 6: Check Response** System checks if user liked it Improve next notification **End**

Algorithm: Change Vital Values

Table 33 Change Vital Values

Algorithm 7: Change Vital Values
Input: Old Vital Data, New Vital Data, User ID
Output: Vital data updated
Step 1: Open Change Option User clicks on "Change Vital" button System shows form to update values **Step 2: Get Current Data** System shows current vital data User can see old values **Step 3: Enter New Values** User types new vital values User checks if all values are correct **Step 4: Save Changes** User clicks "Save" button System updates the data **Step 5: Confirmation** System shows message "Data updated successfully" New values are saved **Step 6: Validation** System checks if input is correct If data is wrong, show "Invalid data" **End**

4.2 External APIs/SDKs

Table 34 External APIs/SDKs

Name of API/SDK	Description	Purpose of usage	Where it is used in system
AI Module	AI chat service	Gives answers, help, and tips to students	AI Chat Bot, Quiz help, Notifications
Fast API (Python)	Backend API framework	Handles all app logic and requests	Login, session, quiz, AI, data handling
MySQL Database	Data storage system	Stores users, vitals, sessions, reports	All system data (users, quiz, reports)
Flutter SDK	Mobile app toolkit	Builds app interface and screens	Full mobile app (UI, graphs, screens)
QR Code Generator	QR code tool	Creates QR codes for sharing vitals/session	Share Vital via QR Code, Session join
QR Code Scanner SDK	Scanner tool	Scans QR code to get data	Join session, view shared vitals
Notification Service (Local/FCM)	Alert system	Sends notifications and updates	Interesting notifications, alerts

4.3 Code Repository

To manage the version control and collaboration effectively for the group project, Git will be used as the primary tool. To make working together easier and keep track of changes in the “MediSim” project. All project files, including code, documentation, and other relevant resources, will be stored in the Git repository.

4.3.1 Git Repository Link

Table 35: Repository Detail

Attributes	Details
Repository name	MediSim
Platform	GitHub
Repository URL	https://github.com/Muhammadzubair0341/medisim-fyp
Repository type	Private (recommendation for academic integrity).
Version Control	Git
Main Branch	Main
Development Branch	dev
Created on	April 2026
Last Updated	Automatically Updated with code commits
Contribute	Backend Developer (Fast API), Frontend Developer (Flutter), Database Designer (MY SQL), Tester.

4.4 Summary

The fifth chapter introduces the basic features of the system MediSim along with its implementation details with the help of Flutter and MySQL Database. It introduces 5 features: User Registration, Quiz System, Emergency scenario, Change Vital Values and Graph and AI Assistant. The whole developing process is organized using the GitHub. We used the branch managing and pull request to promote the cooperative programming.

Chapter 5: Testing and Evaluation

5.1 Testing Evaluation

Full testing was conducted for the MediSim application on its correctness, reliability and performance. It was our main task to ascertain that MediSim application is able to meet its requirements and able to perform on various devices. During the period when the project was confined to limited time, manual testing was required when the resources were limited but test cases were designed based on industry best practices to achieve accuracy.

5.2 Unit Testing

FR-1: Login / Registration

Table 36 UT1: Login / Registration

Test case ID	Description	Input	Expected Output
TC-1.1	User login with correct data	Valid email and password	User logged in successfully
TC-1.2	User login with wrong password	Wrong password	Error: Invalid password
TC-1.3	New user registration	Name, email, password	Account created successfully
TC-1.4	Register with existing email	Email already used	Error: Email already exists

FR-2: Emergency Scenario

Table 37 UT2: Emergency Scenario

Test case ID	Description	Input	Expected Output
TC-2.1	Select emergency case	Choose scenario	Scenario loaded on screen
TC-2.2	Start simulation	Click start	Simulation starts
TC-2.3	Wrong selection	Invalid option	Error message shown
TC-2.4	End simulation	Click stop	Simulation ends

FR-3: Vital Graph Simulation

Table 38 UT3: Vital Graph Simulation

Test case ID	Description	Input	Expected Output
TC-3.1	View vital graph	Select graph option	Graph displayed
TC-3.2	Real-time update	Live data	Graph updates automatically
TC-3.3	No data available	Empty data	Message: No data found
TC-3.4	Change graph type	Select type	Graph style changes

R-4: Change Vital Values

Table 39 UT4: Change Vital Values

Test case ID	Description	Input	Expected Output
TC-9.1	Change value	Enter new value	Value updated
TC-9.2	Invalid value	Wrong input	Error message
TC-9.3	Reset value	Click reset	Default value set
TC-9.4	Save changes	Click save	Changes saved

FR-5: Adaptive Quiz System

Table 40 UT5: Adaptive Quiz System

Test case ID	Description	Input	Expected Output
TC-4.1	Start quiz	Click start	Quiz begins
TC-4.2	Answer question	Select option	Next question shown
TC-4.3	Submit quiz	Click submit	Score displayed
TC-4.4	Skip question	No answer	Next question shown

FR-6: Skill Scoring and AI Report

Table 41 UT6: Skill Scoring and AI Report

Test case ID	Description	Input	Expected Output
TC-5.1	View score	Completed quiz	Score shown
TC-5.2	Generate report	Click report	Report generated
TC-5.3	Low score case	Low marks	Suggestion shown
TC-5.4	High score case	High marks	Good feedback shown

FR-7: AI Chat Bot

Table 42 UT7: AI Chat Bot

Test case ID	Description	Input	Expected Output
TC-6.1	Ask question	Type message	Bot gives reply
TC-6.2	Unknown question	Random input	Bot shows simple answer
TC-6.3	Multiple questions	Many messages	Bot replies one by one
TC-6.4	End chat	Close chat	Chat ends

FR-8: Share Vital via QR Code and ID

Table 43 UT8: Share Vital via QR Code and ID

Test case ID	Description	Input	Expected Output
TC-7.1	Generate QR code	Click generate	QR code created
TC-7.2	Share via ID	Enter ID	Data shared
TC-7.3	Invalid ID	Wrong ID	Error shown
TC-7.4	Scan QR	Scan code	Data opened

FR-9: Interesting Notification

Table 44 UT9: Interesting Notification

Test case ID	Description	Input	Expected Output
TC-8.1	Receive notification	New update	Notification shown
TC-8.2	Open notification	Click message	Details displayed
TC-8.3	No internet	Offline	Notification not loaded
TC-8.4	Multiple alerts	Many updates	All notifications shown

5.3 Integration Testing

Integration Test Case

FR-1: Login / Registration

Table 45 ITC1: Login / Registration

Test ID	**ITC-1.1**
Scenarios	User puts email and password to login. If new, user fills form to make account.
Module Involved	Login page, Sign up form, User data
Expected Results	User logs in or new account is created without problem.

FR-2: Emergency Scenario

Table 46 ITC2: Emergency Scenario

Test ID	**ITC-2.1**
Scenarios	User selects any emergency case from list and starts it.
Module Involved	Scenario screen, Simulation system
Expected Results	Selected case opens and starts on screen.

FR-3: Vital Graph Simulation

Table 47 ITC3: Vital Graph Simulation

Test ID	**ITC-3.1**
Scenarios	User opens graph to see heart rate and other values.
Module Involved	Graph screen, Data system
Expected Results	Graph shows properly and values keep updating.

FR-4: Change Vital Values

Table 48 ITC4: Change Vital Values

Test ID	**ITC-9.1**
Scenarios	User changes values like heart rate.
Module Involved	Control system
Expected Results	Values change and update on graph.

FR-5: Adaptive Quiz System

Table 49 ITC5: Adaptive Quiz System

Test ID	**ITC-4.1**
Scenarios	User starts quiz and answers questions.
Module Involved	Quiz system, Question list
Expected Results	Questions show one by one and answers are saved.

FR-6: Skill Scoring and AI Report

Table 50 ITC6: Skill Scoring and AI Report

Test ID	**ITC-5.1**
Scenarios	After quiz, system checks answers and makes report.
Module Involved	Score system, Report section
Expected Results	User gets marks and simple report is shown.

FR-7: AI Chat Bot

Table 51 ITC7: AI Chat Bot

Test ID	**ITC-7.1**
Scenarios	User types question in chat.
Module Involved	Chat system
Expected Results	System gives simple and clear answer.

FR-7: Share Vital via QR Code and ID

Table 52 ITC8: Share Vital via QR Code and ID

Test ID	**ITC-7.1**
Scenarios	User shares data using QR code or ID.
Module Involved	QR system, Share system
Expected Results	QR is created or ID shares the data.

FR-8: Interesting Notification

Table 53 ITC9: Interesting Notification

Test ID	**ITC-8.1**
Scenarios	System sends updates or alerts to user.
Module Involved	Notification system
Expected Results	Notification comes on time and can be opened.

5.4 System Testing

ST-1: User Registration

Table 54 ST1: User Registration

Test case ID	**ST-1**
Test Scenarios	End-to-end user registration and login
Test Steps	Register --> Email verification --> Login
Expected Output	User account created and logged in successfully

ST-2: Simulation Scenario

Table 55 ST2: Simulation Scenario

Test case ID	**ST-2**
Test Scenarios	Select simulation scenario and start session
Test Steps	Login --> Open scenarios --> Select case --> Start
Expected Output	Simulation scenario loaded successfully

ST-3: Vital Graph

Table 56 ST3: Vital Graph

Test case ID	**ST-3**
Test Scenarios	Real-time vital graph simulation
Test Steps	Start scenario --> Load vitals --> Show ECG/SPO2/BP graphs
Expected Output	Live graphs displayed smoothly

ST-4: Change Vital Values

Table 57 ST4: Change Vital Values

Test case ID	**ST-4**
Test Scenarios	Instructor changes vital values during session
Test Steps	Login as Instructor --> Open session --> Change HR/BP/SPO2
Expected Output	Updated values shown to all users in real time

ST-5: AI Chat Bot

Table 58 ST5: AI Chat Bot

Test case ID	**ST-5**
Test Scenarios	AI Chat Bot support during training
Test Steps	Login --> Open AI Chat --> Ask medical question
Expected Output	AI gives helpful response successfully

ST-6: Adaptive Quiz System

Table 59 ST6: Adaptive Quiz System

Test case ID	**ST-6**
Test Scenarios	Adaptive quiz starts and completion
Test Steps	Login --> Open quiz --> Answer questions --> Submit
Expected Output	Quiz completed and score displayed

ST-7: Skill Scoring and AI Report

Table 60 ST7: Skill Scoring and AI Report

Test case ID	**ST-7**
Test Scenarios	Skill scoring and AI report generation
Test Steps	Finish quiz/simulation --> System checks results --> Generate report
Expected Output	Score and report generated successfully

ST-8: Session Creation

Table 61 ST8: Session Creation

Test case ID	**ST-8**
Test Scenarios	Session creation and QR code join
Test Steps	Instructor creates session --> Generate QR --> Student scans QR
Expected Output	Student joined session successfully

ST-9: Share Vital via QR Code and ID

Table 62 ST9: Share Vital via QR Code and ID

Test case ID	**ST-9**
Test Scenarios	Share vital signs via QR code and ID
Test Steps	Open share option --> Generate QR/ID --> Other user scans/enters ID
Expected Output	Shared vital data opened successfully

ST-10: Notification System

Table 63 ST10: Notification System

Test case ID	**ST-10**
Test Scenarios	Notification delivery and open
Test Steps	System sends medical fact alert --> User taps notification
Expected Output	Notification opened

5.5 Performance Testing

PT-1 User Registration

Table 64 PT1: User Registration

Test ID	**PT-1**
Test Scenario	User login check
Input Data	50 user's logins at the same time
Expected Output	Users' login in 3 seconds
Actual Output	Users logged in after 2.7 seconds
Result	Pass

PT-2: User Activation

Table 65 PT2: User Activation

Test ID	**PT-2**
Test Scenario	Open training case
Input Data	30 users open one case together
Expected Output	Case opens in 4 seconds
Actual Output	Case opened in 3.1 seconds
Result	Pass

PT-3: Vital Graph

Table 66 PT4: Vital Graph

Test ID	**PT-3**
Test Scenario	Vital graph check
Input Data	20 users run ECG, SPO2 and BP graphs
Expected Output	Graphs work without stop
Actual Output	Graphs worked smoothly
Result	Pass

PT-4: Chat Bot Performance

Table 67 PT4: Chat Bot Performance

Test ID	**PT-4**
Test Scenario	Chat bot speed check
Input Data	25 users send questions together
Expected Output	Reply comes in 5 seconds
Actual Output	Reply came in 4.2 seconds
Result	Pass

PT-5: Quiz Performance

Table 68 PT5: Quiz Performance

Test ID	**PT-5**
Test Scenario	Quiz open check
Input Data	40 users open quiz together
Expected Output	Quiz opens in 3 seconds
Actual Output	Quiz opened in 2.5 seconds
Result	Pass

PT-6: QR Code and ID Sharing

Table 69 PT6: QR Code and ID Sharing

Test ID	**PT-6**
Test Scenario	QR code speed check
Input Data	20 users make QR code
Expected Output	QR code ready in 2 seconds
Actual Output	QR code ready in 1.4 seconds
Result	Pass

PT-7: Report Creation

Table 70 PT7: Report Creation

Test ID	**PT-7**
Test Scenario	Report create check
Input Data	15 users make reports
Expected Output	Report ready in 5 seconds
Actual Output	Report ready in 3.8 seconds
Result	Pass

PT-8: Joining Sessions

Table 71 PT8: Joining Sessions

Test ID	**PT-8**
Test Scenario	Join with QR code
Input Data	35 students scan one QR code
Expected Output	All students join easily
Actual Output	Students joined in 3.6 seconds
Result	Pass

PT-9: Change Vital Values

Table 72 PT9: Change Vital Values

Test ID	**PT-9**
Test Scenario	Change live values
Input Data	Teacher changes values for 25 users
Expected Output	New values show fast
Actual Output	Values changed in 1.2 seconds
Result	Pass

5.6 Summary

This chapter presents all of the test cases which is a document used to test the testing stage of the MediSim application. The tests are written to meet functional and non-functional requirements of the application. The types of testing that can be performed as described in this document is function testing, unit testing, integration testing and performance testing. The testing performed showed the system behaves in a desirable way under ideal circumstances but there is room for improvement when pushed to the extreme.

Chapter 6: System Conversion

6.1 System Conversion

System conversion is the changeover from the old manual system to the new digital system. In the MediSim project the old way of learning such as reading books and simple class practice is converting into an easy mobile application. MediSim allows users to practice emergency cases, check vitals and take quizzes all in one place. The system is built in a way that users can understand and use it without any problem. MediSim main goal is to provide a better and easier way of learning so users can improve their skills any time using their mobile.

6.2 Conversion Method

Several conversion strategies can be employed depending on the size and complexities of the MediSim. For MediSim, the following methods are considered:

6.2.1 Direct Conversion

- **Description:** In this method, the old system is completely removed and the MediSim system is started immediately.
- **Applicability:** It is a procedure that can be applied to small systems. It can be used for testing or for a small group of users in MediSim.

6.2.2 Parallel Conversion

- **Description:** With this approach the old system and MediSim are both running for a while. The results from both systems are compared to check whether everything is working correctly.
- **Applicability:** MediSim app is giving correct results, vital signs simulation and reports.

6.2.3 Phased Conversion

- **Description:** This method describes the MediSim system step by step. Features are launched one by one. For example, login system first, emergency scenarios second, quiz system third etc.
- **Applicability:** This is better for medium and larger scale projects like MediSim It helps to easily handle errors and gives time to improve each module before moving to the next one.

6.2.4 Pilot Conversion

- **Description:** MediSim is first introduced to a limited number of users, such as one class or a few students. Then it is tested successfully and introduced to all the users.

- **Application:** MediSim it allows for real user feedback. It helps you to fix problems early, before you release the app to everyone.

6.3 Deployment

The MediSim application will be distributed to users manually or through an app store, the following steps outline how to build and deploy the app for Android and in future iOS or a web application.

Prerequisites

- Flutter SDK installed
- Android Studio or command-line Android SDK (for Android builds)
- Real devices or emulators for testing

Android Deployment

Table 73 Android Deployment Steps

Step	Description
Configure App	Update android/app/build.gradle with versionCode, versionName, applicationId, etc.
Run Build Command	Run: Flutter build APK This will generate a release APK at: build/app/outputs/flutter-apk/app-release.apk
Test on Device	Transfer the APK to an Android phone or use Android Studio's emulator to install and test.
Distribute APK	• You can distribute the APK manually or upload it to the **Google Play Store** • Sign the APK • Create a developer account • Upload via Google Play Console

Docker Deployment

Table 74 Docker Deployment

Step	Description
Install Docker	Ensure Docker is installed on your local development system.
Prepare the Mobile Project	Make sure the mobile app (e.g., Android Flutter app) is structured correctly with all dependencies declared (e.g., in pubspec.yaml for Flutter or package. Json for React Native).
Create a Docker file	Write a Docker file that uses a base image with Android SDK or Flutter SDK installed. This file will define how to build the APK within a containerized environment.
Build the Docker Image	Run the Docker build command: docker build -t Medisim-mobile
Run Container to Build APK	Use the Docker container to generate the APK (Android Package): docker run -v $flWD:/app medism-mobile. /gradlew assembleDebug (for native) or flutter build apk (for Flutter).
Retrieve APK	After the build completes, retrieve the generated APK file from the container- mounted volume (e.g., in the build/outputs/apk/ directory).
Manual Installation	The APK can be transferred to Android devices for testing or shared with users. For wider distribution, it can be published on Google Play after signing and verification.
Test and verify	Install and check the APK in actual android devices and emulators that all modules and features of the MediSim would work fine such as Vital sign & Graph, Quiz system, Generated Report, Chat bot, login.

6.4 Data Conversion

Since MediSim uses the newer database system, it deals with data like this on its own, therefore it was not necessary to convert old data from the legacy system. As MediSim application is stored with the structured database (e.g. MySQL) it requires no manual transition of records from the old system. Nevertheless, some general procedures may be required to smoothly deploy MediSim.

Table 75 Data Conversion Plan

Steps	Description
Create Database Schemas	When first MediSim used the structure of the database is automatically created, then, when the user adds new data like profile or view graph, starts chat, etc. Is sorted in DB without setting anything manually.
Insert Starter Data (optional)	Some initial data, graph, or vital signs, or app settings can be added in beginning. This process is so simple with the MYSQL Structure.
Data Safety and Recovery	The data can be backup in JSON file so it may restore if lost.
Verify Information	The system checks all the data entered by users using built-in validation rules. This helps keep the database accurate and reduces mistakes.

6.5 Training

Training Plan (MediSim):

All users will be given a training session on the appropriate and simple use of the MediSim to ensure everyone can make use of it.

Training includes:

- This training is for students.
- How to create an account and log in to the system
- How to start and handle emergency scenarios
- Understanding vital signs and graph simulation
- Attempting quizzes and checking results
- Viewing performance reports and scores
- Using the AI chat feature for help and guidance

Training Format:

The training session will be of practical nature, and user friendly. It involves actual use of the system, with users navigating the system step-by-step.

Estimated Duration:

The training can be completed in 1 day, but it may take up to 2 days depending on the user's understanding and experience.

6.6 Post Deployment Test

After deploying the MediSim mobile application, post-deployment testing is essential to ensure that the application is functioning correctly, the system is fully operational, and all modules are properly integrated. The following steps outline the testing process:

- Verify that the application installs and launches successfully on Android devices.
- Test user registration, login, and authentication with MYSQL to ensure account handling is functional.
- Validate that user can change view graph and vital signs, share qr code or id, attempts quiz, generate report, communicate to ai chat bot.
- Check database connectivity and confirm that data is being stored and retrieved correctly from MYSQL and Storage.
- Verify that appropriate error messages are shown for invalid actions.

6.7 Challenges

Learning of new technologies was one major difficulty our team experienced during the project. Since we are bound to use Flutter and Dart to create this mobile application. We had to leave the comfort zone. Learning of Flutter and Dart took much effort and strenuous commitment. We spent additional time outside our scheduled classes and join a training course to develop the aptitude needed in our project schedule.

Another of the key problems was the choice of a suitable database solution. We came up with the decision to use MYSQL after exploring several possibilities taking into consideration real time data and scalability. However, this decision also meant that we had to learn how to valuably cooperate with MYSQL that was new to us initially.

6.8 Summary

The chapter, details the whole conversion and deployment process of MediSim mobile application. The section includes selection of the conversion process, deployment of Flutter android builds, integrating MYSQL and handling data, requiring minimum user training, etc.

Chapter 7: Conclusion

7.1 Conclusion

The MediSim project successfully achieve its objective of developing a unified Graph Simulation System Platform that integrates multiple professional functionalities into a single system. The platform addresses the limitation of existing system by combining a Graph Simulation, Vital Values, Quiz System, AI Chat Bot and Notification System with in secure and scalable environment.

The system is developed using a **Fast API (Back-end)** of a **Python framework** and **Flutter (Front-end)** for responsive design ensuring modern development practices, maintainability and performance. Secure user registration is implemented using email-based OTP verification.

Overall, the system provides a complete digital ecosystem that provide efficiency, transparency and interaction among Users.

7.2 Evaluation

Table 76 Evaluation

Objective	Status
Allow users to create an account and log in	Completed
Create different emergency situations for user practice.	Completed
Show graphs like heart rate.	Completed
Provide quiz system to check user.	Completed
Create a simple report based on user activity.	Completed
Provide a chat option to help users when needed.	Completed
Save user data safely in the database.	Completed
Keep the system simple and easy to use for everyone.	Completed

7.3 Traceability Matrix

Table 77 Traceability Matrix

Requirement ID	Requirement Description	Design Specification
FR-1	User can sign up, login and manage their profile	Login system using username, password, and dashboard
FR-2	Users can access different emergency scenarios for practice.	Scenarios are shown in the system with easy navigation.
FR-3	System shows graph.	Graph feature to display vitals values and graphs in real time.
FR-4	Users can attempt quizzes to test their knowledge.	Quiz module with questions and answer options.
FR-5	System gives marks based on user performance.	Simple scoring system after quiz or simulation.
FR-6	System creates a report based on user activity.	Report section to show user progress and results.
FR-7	Users can use chat option for help.	Chat feature added for user support.
FR-8	User data is saved in the database.	Data stored safely using database system.
FR-9	Users receive notifications for updates and activities.	Notification system to alert user and important updates.

7.4 Future Work

These further enhancements focus on making the system more intelligent, expandable and user friendly. Currently the system is fully functional, but with some possible extensions and improvements to make:

- Multi-Scenario Improvement: More emergency scenario can be added.
- Advanced AI Support: Chat system can be improved to give more accurate guidance.
- Web Version Upgrade: The system can be enhanced with a more interactive and responsive web interface.

Appendix A

Use Case Description

Use Case Analysis

Use Case Analysis is a software engineering technique used to identify and describe the interactions between system actors and the system itself. It helps define functional requirements by explaining how different users perform specific tasks within the system to achieve particular goals.

Use Case # 1

UC Identifier	MS-UC1
Functional Requirements	Register User
Actor	Student/Instructor
Preconditions	The user must be on the sign-up screen.
Post Conditions	The user is registered and can now log in to the app.
Description	The user arrives at the registration screen, and taps Sign Up. They enter the name, email, and password. Everything is checked, and a verification code is fired off to the email. The user grabs it from the email and enters it in the app, after which their account is created and the user is directed straight to the login screen.

Use Case # 2

UC Identifier	MS-UC2
Functional Requirements	Select Simulation Scenario
Actor	Student/Instructor
Preconditions	The user must be logged in and on the main dashboard.
Post Conditions	The chosen scenario is loaded and ready to start.
Description	• User clicks "Start Training" or "Scenarios" button • The system will then show a list of available case scenarios to practice (e.g. Heart Attack, Asthma or ER Emergency). • User reads brief description of the case scenario • User picks the scenario they would like to practice • The system then creates the virtual patient with the correct vitals • User clicks the "Start" button.

Use Case # 3

UC Identifier	MS-UC3
Functional Requirement	AI Chat Bot
Actor	User
Preconditions	The user must have logged into the system.
Post Conditions	The student gets the right answers to their queries.
Description	• User open chatbot • Type medical queries • System process queries • Ai generates response displayed to the user

Use Case # 4

UC Identifier	MS-UC4
Functional Requirement	Generate Performance Report
Actor	Student/Instructor
Preconditions	The user must have finished a simulation session.
Post Conditions	A report is created and shown on the screen for the user to read.
Description	The program stops after a set time, or when the user selects to "Finish". The system takes a data file from the session containing all the relevant results such as accuracy and reaction time. The program gives the user a score based on the results of the session. A report page appears displaying the areas the user performed well in, as well as where the errors were made. The user then views the final score or some helpful advice for the next time the program is used. The user then selects Save or Close and returns to the main menu.

Use Case # 5

UC Identifier	MS-UC5
Functional Requirement	Adaptive Quiz System
Actor	Student
Preconditions	User must be registered and logged into the MediSim mobile application.
Post Conditions	Quiz results are saved; performance is evaluated.
Description	• Student selects adaptive quiz. • System loads initial questions. • Student answers questions. • System evaluates answers in real-time. • Quiz continues based on performance. • Final score and feedback are displayed. • Results are stored in database.

Use Case # 6

UC Identifier	MS-UC6
Functional Requirement	Notification system
Actor	Student
Preconditions	User must be signed-in into the MediSim system.
Post Conditions	User receive accurate and relevant medical facts in the form of notifications.
Description	Ai randomly selects the interesting facts and information about the medical field specially EMS facts and information and sends the notification to the user.

Use Case # 7

UC Identifier	MS-UC7
Functional Requirement	Skill Scoring and Performance Monitoring
Actor	Student
Preconditions	User must be signed-in into the MediSim system
Post Conditions	The system generated and display a performance score and report.
Description	• Student completes a simulation. • System checks all performance. • System calculates of quizzes. • Performance score is calculated. • System generate report of all performances and quiz.

Use Case # 8

UC Identifier	MS-UC8
Functional Requirement	Session Creation and QR Code and ID
Actor	Student/Instructor
Preconditions	User must be signed-in into the MediSim system
Post Conditions	A new simulation session is created and a QR code and id is generated to fast join the session.
Description	• Creation of a session • Share session via QR code

Appendix B
Coding Standards

In order to maintain the quality, the scale and the readability of code in development of MediSim, the developer team is using following (The standards of the code styles of all the tools are based on these):

1. Variable Naming Conventions

- **camelCase** is employed for both frontend (Flutter) and backend (Fast API) variables and local functions.
- **Pascal Case** used for class names, DTOs, services, controllers and interfaces
- **UPPER_CASE** are constants/configuration value such as MAX_FINE_AMOUNT, JWT_SECRET_KEY.

2. Function Naming

- Functions & Methods: They are descriptive and action-oriented (e.g. Generate Report(), Validate Login(), Mark Verify OTP()).
- The controller methods have names corresponding to RESTful operations.
- Boolean functions start with a verb (like IsUserValid(), HasPermission()).

3. Code Comments

- Comments are applied to explain intricate business logic in services, APIs, and database transactions.

4. Indentation and Formatting

- Standard indentation is followed across all frontend and backend codebases.
- It is 4-space indentation. Loops, conditions, methods have got Consistent spacing.

5. File and Foder Naming

- **Front end:** Pascal Case is used for all the Dart screen, widget and component file.
- **Back-end:** Organize into layer:
    - Routes
    - Services
    - Controller
    - Schemas
    - Utility
    - Models

- Database Configuration

6. Code Structure and Modularity

MediSim is based on Clean Architecture where the concerns are separated.

- **Presentation Layer:** Flutter (UI components, pages)
- **Application Layer:** Business Logic (Services)
- **Domain Layer:** Core entities and interfaces
- **Infrastructure Layer:** Database, external services

7. Access Modifier

- Private for internal logic
- Public for API and Services
- Protected for inheritance-based logic
- Interfaces are used to hide implementation detail and enforce abstraction

8. Extensibility

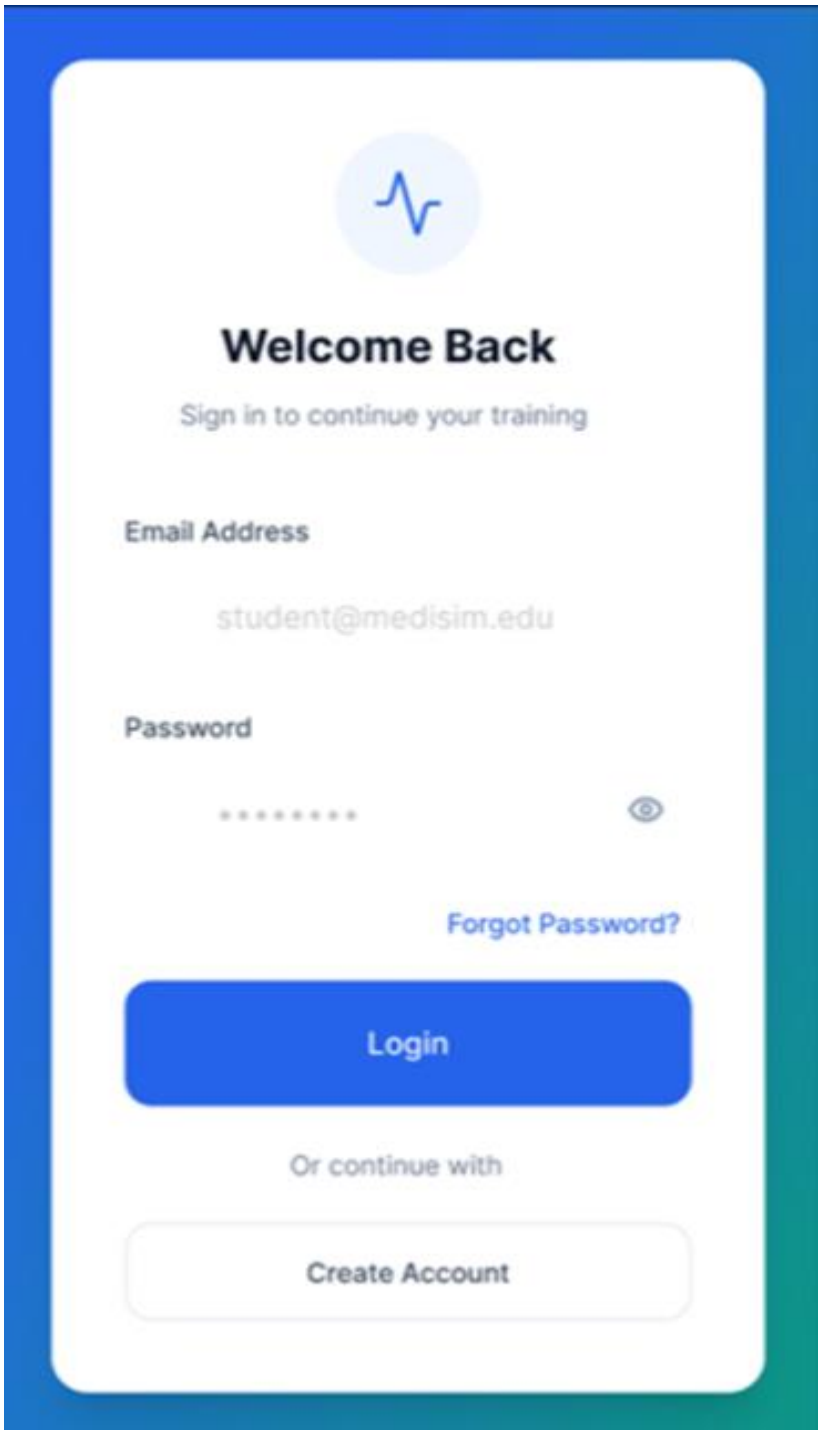
- System is designed to support future expansion without major changes.
- New modules (Added more Scenarios, AI-based analytics, advance AI Chat Bot).
- With a microservice ready architecture, each module can be scaled independently.

9. Exception Handling

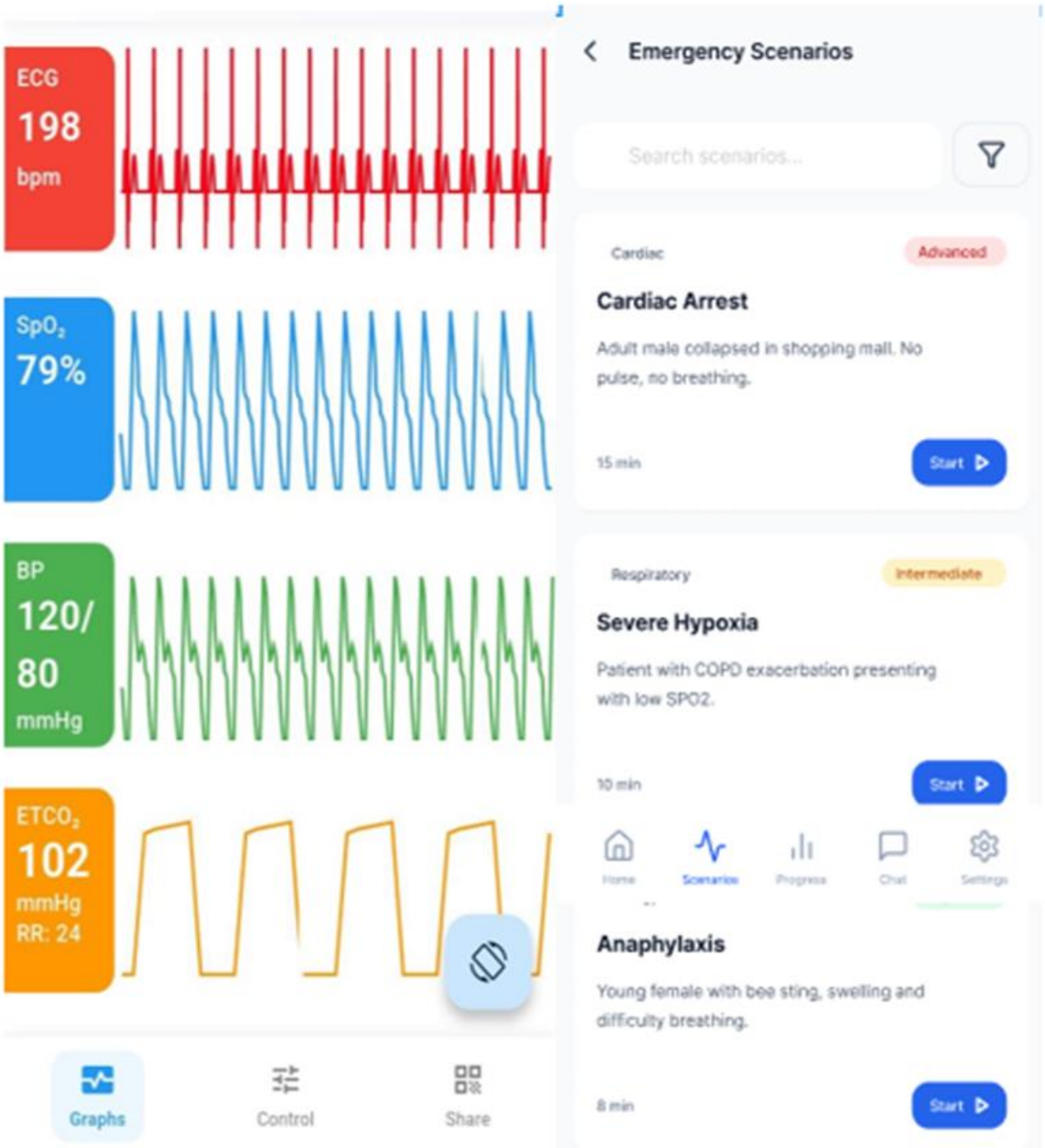
- All API endpoints include proper try-catch blocks for error handling.
- lobal exception middleware is implemented in .NET Core.
- User-friendly error responses are returned to frontend applications.
- Validation errors are handled using model validation attributes.

Appendix C
Application Prototype

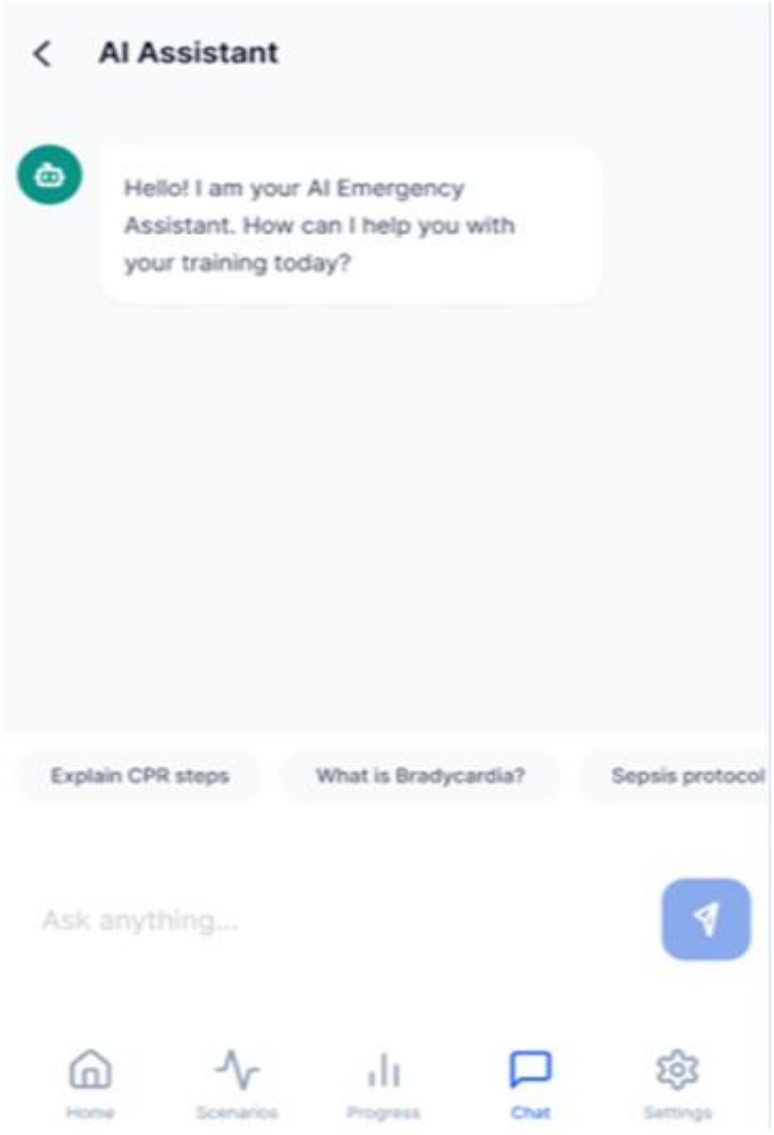
1. Login Screen



2. Simulation Graphs



3. AI Chat Bot


AI Assistant
Hello! I am your AI Emergency Assistant. How can I help you with your training today?
Explain CPR steps
What is Bradycardia?
Sepsis protocol
Ask anything...
Home
Scenarios
Progress
Chat
Settings

4. Performance Report



5. QR Code and ID

